

本科毕业论文（设计）

基于大模型驱动的 Agent 协作新一代蛋白质设计系统开发

Development of a Next-Generation Protein Design System Based on Large-Scale Model-Driven Agent Collaboration

郑彦文

哈尔滨工业大学

2026 年 6 月

☐毕业论文 ☐毕业设计

密级：公开

本科毕业论文（设计）

基于大模型驱动的 Agent 协作新一代蛋白质设计系统开发

本 科 生： 郑彦文

学 号： 2022112879

指 导 教 师： 陈源龙

专 业： 软件工程

学 院： 计算学部

答 辩 日 期： 2026 年 5 月 26 日

学 校： 哈尔滨工业大学

摘 要

蛋白质计算设计通常需要将序列生成、结构预测、质量控制、目标评分和结果汇总等环节组织为连续工作流。现有工具已有较强单点能力，但实际任务仍面临接口异构、输入输出约束复杂、高代价步骤不宜盲目调用、运行时失败需要恢复、关键决策需要人工审阅等问题。针对这些问题，本文设计并实现了一个面向蛋白质设计任务的大模型驱动多 Agent 协作与自适应工作流规划原型系统，在已有工具和远程模型服务之上构建可规划、可执行、可恢复和可审计的工作流控制框架。

系统按输入交互、智能规划、工作流执行、安全汇总和资源管理进行分层，用 ProteinToolKG 描述工具能力和 I/O 兼容关系，用 FSM 约束任务生命周期，并通过 HITL 处理高风险、高成本或不确定性较强的候选选择。本文提出并实现 CEBRA-WP 工作流规划算法，将任务约束、工具知识、执行历史、运行时观测和 Lite belief-state / 轻量信念状态用于候选过滤、评分、重排序和恢复动作选择。系统测试覆盖 13 个用例，其中 12 个通过、1 个 CLI 相关用例部分通过；84 runs 内部消融实验得到 81 个 DONE 和 3 个 FAILED。结果表明，该原型系统能够在工作流层支撑任务接入、候选规划、人工确认、失败恢复和证据审计，但不等同于真实生物学功能验证。

关键词：蛋白质设计；大模型驱动；多 Agent 协作；自适应工作流规划；人在环决策

Abstract

Computational protein design often needs to connect sequence generation, structure prediction, quality control, objective scoring, and result summarization into a continuous workflow. Existing tools already provide useful capabilities for individual steps, but practical tasks still involve heterogeneous interfaces, complex input-output constraints, high-cost calls, runtime failures, and human review for critical decisions. To address these workflow-level problems, this thesis designs and implements an LLM-driven multi-Agent collaboration and adaptive workflow planning prototype for protein design tasks. The system aims to build a plannable, executable, recoverable, and auditable workflow control framework on top of existing tools and remote model services. It uses a layered architecture for input interaction, intelligent planning, workflow execution, safety summarization, and resource management. ProteinToolKG describes tool capabilities and I/O compatibility, FSM constrains the task lifecycle, and HITL handles candidate selection under high risk, high cost, or strong uncertainty.

To support runtime recovery, this thesis proposes and implements CEBRA-WP, which combines task constraints, tool knowledge, execution history, runtime observations, and a Lite belief-state for candidate filtering, static scoring, posterior objective adaptation, runtime reranking, and recovery action selection. The prototype is implemented with Python, FastAPI, Pydantic, React, TypeScript, and Vite, while ToolAdapter encapsulates capabilities such as structure prediction, sequence generation, quality control, and objective scoring. System validation covers 13 test cases, with 12 passed and one CLI-related case partially passed. An internal 84-run ablation matrix produces 81 DONE runs and 3 FAILED runs. The results show that the prototype can support task intake, candidate planning, human confirmation, failure

recovery, and evidence auditing at the workflow layer. The conclusion is limited to engineering-prototype and workflow-mechanism validation, rather than wet-lab biological validation.

Keywords: protein design, llm-driven system, multi-agent collaboration, adaptive workflow planning, human-in-the-loop

目 录

摘 要	I
Abstract	II
第 1 章 绪 论	1
1.1 课题背景及研究的目的和意义	1
1.2 蛋白质设计技术发展概述	2
1.2.1 蛋白质计算设计的发展	2
1.2.2 深度学习驱动的蛋白质设计工具	2
1.2.3 工作流自动化需求	3
1.3 决策理论与 Agent 协作	4
1.3.1 马尔可夫决策过程的发展	4
1.3.2 部分可观测决策过程	4
1.3.3 大语言模型 Agent 协作	5
1.4 本文的主要研究内容	6
第 2 章 相关技术与理论基础	8
2.1 蛋白质计算设计基础	8
2.2 蛋白质设计与分析工具	8
2.3 科学工作流与可复现执行	9
2.4 大语言模型工具调用	10
2.5 部分可观测规划与预算感知决策	10
2.6 相关系统与本文定位	11
2.7 本章小结	12
第 3 章 需求分析	13
3.1 问题界定	13

3.2 用户角色与使用场景	14
3.3 功能需求	15
3.3.1 任务接入与约束确认	15
3.3.2 候选计划生成与工具链选择	15
3.3.3 workflow 执行与工具适配	16
3.3.4 HITL 与决策审查	16
3.3.5 运行时自适应与恢复决策	16
3.3.6 结果汇总、报告与审计	17
3.4 非功能需求	17
3.4.1 可追溯性	17
3.4.2 可扩展性	17
3.4.3 可靠性与可恢复性	18
3.4.4 安全与边界控制	18
3.4.5 实验可复现性	18
3.4.6 需求优先级与边界	18
3.5 本章小结	19
第 4 章 系统设计	20
4.1 设计目标与总体架构	20
4.2 核心组件与职责边界	22
4.3 任务生命周期与 HITL 约束	23
4.4 六阶段蛋白质设计 workflow	26
4.5 CEBRA-WP 的提出背景	27
4.6 CEBRA-WP 形式化定义	27
4.7 候选过滤与静态效用	31
4.8 轻量信念状态与观测更新	33
4.9 后验目标评分与运行时重排序	35

4.10 恢复动作与 HITL 映射	36
4.11 策略组与实验可验证性	39
4.12 数据契约与模块协作	40
4.13 本章小结	42
第 5 章 系统实现	43
5.1 技术选型与工程结构	43
5.2 任务接入与后端 API 实现	45
5.2.1 渐进式任务录入	45
5.2.2 任务生命周期接口	47
5.2.3 HITL 接口	47
5.3 前端工作台实现	48
5.4 运行时与执行引擎	48
5.4.1 WorkflowContext	49
5.4.2 计划与步骤执行器	50
5.4.3 等待状态、审计与快照	52
5.5 CEBRA-WP 的工程落点	54
5.6 工具适配器与能力管理	57
5.7 本章小结	59
第 6 章 系统测试与验证	60
6.1 测试策略与验证目标	60
6.2 API 服务与任务录入验证	63
6.3 计划候选与 HITL 验证	64
6.4 状态机、快照与恢复验证	64
6.5 前端与 CLI 可用性验证	65
6.6 失败恢复与安全验证	66
6.7 端到端与工具链验证	68

6.8 本章小结	69
第 7 章 实验与结果分析	70
7.1 实验设计	70
7.2 指标定义	72
7.3 四组消融主结果.....	73
7.4 分层结果与机制增量.....	75
7.5 成本与恢复行为分析	76
7.6 轻量信念状态可观测性	78
7.7 典型失败案例	78
7.8 证据产物与本章结论	80
结 论	81
参考文献	83
附录 A 系统验证与测试明细.....	87
哈尔滨工业大学本科毕业论文（设计） 原创性声明和使用权限	92
致 谢	94

第1章 绪 论

1.1 课题背景及研究的目的和意义

蛋白质是生命活动中的核心功能分子，参与催化反应、信号传导、物质运输、免疫识别和细胞结构维持等过程。其功能通常与氨基酸序列、三维结构、局部相互作用模式以及分子结合关系密切相关。传统实验方法可以测定和验证蛋白质结构与功能，但在候选设计、筛选和优化阶段往往周期较长、成本较高。如何借助计算方法生成、筛选和评估候选蛋白质序列，因而成为计算生物学、结构生物学和人工智能交叉研究中的关键问题。

蛋白质计算设计主要面向结构、稳定性或功能要求生成候选序列。早期方法依赖结构模板、能量函数和构象采样，Rosetta 等平台可以看作代表性工程形态^[1]。近年来，AlphaFold、AlphaFold 3、ProteinMPNN、ProtGPT2、ESMFold 和 RFdiffusion 等方法分别促进了结构预测、逆向折叠、序列生成和结构生成发展^[2-7]，使多工具协同逐步成为蛋白质设计流程的常见组织形式。

但单个模型变强并不意味着任务可端到端自动完成。在实际设计中，仍然需要经历目标理解、约束整理、候选生成、结构预测、质量检查、目标评分和失败恢复。不同工具在接口、成本和失败模式上差异比较明显；若任务开始时固定工具链，中间质量不足、预算升高或需要人工确认时，系统很难及时调整后续计划。

大语言模型 Agent（Large Language Model Agent, LLM Agent）为自然语言任务解析和工具调用提供了一种可用入口。ReAct 将推理与行动交替组织^[8]，Toolformer 说明语言模型可以学习调用外部工具^[9]。但在本文场景中，蛋白质设计具有高成本、强约束和长链路特征，还需要显式工具约束、状态控制、人工确认和恢复机制共同约束 Agent 行为。

本文面向“基于大模型驱动的 Agent 协作新一代蛋白质设计系统开发”，目标是在已有蛋白质设计工具和 LLM Agent 能力之上，构建可执行、可追踪、

可恢复的 *de novo* 蛋白质设计 workflow 系统。其价值在于统一组织分散工具，降低工具选择和结果追踪负担；用工具知识图谱、FSM、HITL 和事件日志提升可控性；并以 CEBRA-WP 描述高代价、部分可观测 workflow 中的受控规划过程。

1.2 蛋白质设计技术发展概述

1.2.1 蛋白质计算设计的发展

蛋白质计算设计是在目标结构或功能约束下寻找合适氨基酸序列。序列空间巨大，序列、结构与功能之间映射复杂，穷举难以奏效。传统方法多将问题转化为能量优化、构象采样或结构匹配，具有较好解释性，但在复杂功能、多目标约束场景下仍依赖大量人工经验。

深度学习模型进入结构预测和序列建模后，蛋白质计算设计逐渐形成模型驱动路线。结构预测模型可以根据序列推断三维结构，帮助设计者较快判断候选序列是否可能折叠为合理构型；逆向折叠模型可以根据给定骨架生成与结构相容的序列；生成式模型则能在更大范围探索天然蛋白质序列或结构空间。因此，蛋白质设计已经不再只依赖单一能量函数和人工迭代，而更像是多模型、多工具、多指标共同参与的计算流程。

模型能力进入系统后会遇到更具体的工程问题。不同模型的任务、输入输出、资源需求和失败模式并不完全一致；候选序列通常还需清洗、预测、质检和评分。一旦中间步骤失败，系统需要判断是局部修补、替换后续工具，或是终止当前候选。

1.2.2 深度学习驱动的蛋白质设计工具

就工具能力而言，蛋白质设计模型目前大致分布在结构预测、逆向折叠、序列生成和结构生成几类任务上。结构预测方面，AlphaFold、AlphaFold 3、ESMFold 和 OpenFold 是常见代表。AlphaFold 提高了结构预测结果的可靠性，使预测结构可以作为候选序列评估的重要证据^[2]；AlphaFold 3 将预测对象扩展到更复杂的生物分子相互作用场景^[3]；ESMFold 借助蛋白质语言模型进行结构预测，在速度和部署便利性上具有优势^[6]；OpenFold 则提供了 AlphaFold2 的开源实现和工程化再训练基础，便于系统集成和本地部署^[10]。

逆向折叠主要解决“给定骨架，设计相容序列”的问题。**ProteinMPNN** 是结构条件序列设计的代表^[5]；**ProtGPT2** 说明蛋白质序列可作为符号序列进行生成建模^[4]；**RFdiffusion** 则从扩散生成角度支持 **de novo** 结构与功能设计^[7]。

上述工具构成了本文系统的重要能力来源，但本文并不重新训练或替代这些模型，而是讨论如何把异构能力纳入同一工作流。对系统而言，模型需要被描述成可调用工具，并明确工具名称、输入字段、输出字段、成本等级、风险等级和适用任务。只有完成这种结构化表示，**Agent** 才能据此生成候选计划，执行器才能按统一契约调用工具，安全组件也才能在高风险步骤前进行拦截或转入人工确认。

1.2.3 工作流自动化需求

蛋白质设计任务天然带有明显的工作流特征。一次完整流程往往不是单次模型推理，而是多个工具相互配合的过程：自然语言目标要被转换为结构化约束，候选工具链要通过输入输出闭包检查，执行过程中还要根据中间结果调整后续计划，最后再汇总序列、结构、指标、风险和日志，形成便于复核的报告。

传统科学工作流系统更强调任务编排、可复现执行和数据追踪，**Nextflow** 等工作体现了工作流管理在科学计算中的价值^[11]。但蛋白质设计任务当中还存在更强的不确定性和交互性：用户目标常以自然语言给出，初始约束可能并不完整；候选工具链也不适合固定不变，而要结合预算、质量指标和风险等级进行调整；结构预测、目标评分等步骤成本较高，证据不足时需要避免盲目调用；高风险或不可逆操作还需要人工确认，不能完全交给自动执行模块处理。

基于这些特性，本文把蛋白质设计工作流自动化的理解为“受约束的智能工作流规划与执行”问题。系统既利用 **LLM Agent** 的自然语言理解和候选生成能力，也通过工具知识图谱、有限状态机和人在环机制约束 **Agent** 行为。除自动执行外，事件日志、快照和恢复历史也需要被保存下来，使执行过程可以追踪、复核和用于实验分析。这一需求是后续系统设计和算法设计的基本出发点。

1.3 决策理论与 Agent 协作

1.3.1 马尔可夫决策过程的发展

马尔可夫决策过程（Markov Decision Process, MDP）为序贯决策问题提供了基本框架。它通常把决策过程表示为状态、动作、状态转移和奖励函数，并用策略描述不同状态下的动作选择。Bellman 关于马尔可夫决策过程的工作为后续研究奠定了动态规划和序贯决策优化重要基础^[12]，Puterman 则较为系统地整理了离散随机动态规划和 MDP 理论^[13]。在完全可观测环境中，MDP 可以描述系统如何依据当前状态选择动作，并通过长期收益优化得到较优策略。

蛋白质设计 workflow 同样包含类似的序贯决策结构。系统需要在候选生成、结构预测、质量检查、目标评分和恢复处理之间不断选择下一步动作；每一次选择都会改变任务状态，并影响后续成本、风险和结果质量。例如候选序列质量不足时，可以局部修补、重新生成候选，也可以终止当前路径；预算消耗较高且证据不足时，也需要判断是否继续进入结构预测等高代价步骤。因此，MDP 中“状态—动作—转移—收益”的思想有助于刻画本文的工作流控制问题。

但在本文场景中，蛋白质设计 workflow 并不适宜直接写成标准 MDP。系统无法完全观察真实状态，也难以准确获得所有状态转移概率和奖励函数。候选序列是否真正具有生物学有效性、后续工具能否顺利运行、当前失败原因是否可恢复，往往只能通过中间指标和工具反馈间接判断。因此，本文需要借鉴部分可观测决策理论。

1.3.2 部分可观测决策过程

部分可观测马尔可夫决策过程（Partially Observable Markov Decision Process, POMDP）将状态不可完全观测的情况纳入决策模型。与 MDP 假设状态可直接获得不同，POMDP 认为系统只能获得与真实状态相关的观测，因此需要维护 belief-state / 信念状态，用以表示系统对隐含状态的估计。Sondik 早期研究了部分可观测马尔可夫过程在无限时域折扣成本下的最优控制问题

[14], Monahan 对 POMDP 的理论、模型和算法进行了综述[15], Kaelbling 等进一步系统阐述了部分可观测随机领域中的规划与行动问题[4]。

POMDP 对本文的启发在于：在不确定环境中，系统不应只根据单个观测结果作出机械判断，而应综合历史、当前观测和任务目标形成对状态的估计。对应到蛋白质设计 workflow 中，系统看到的只是工具返回结果、错误信息、质量指标、预算消耗和人工决策，并不能直接知道候选路径是否一定成功。因此，workflow 规划器需要根据这些运行时证据判断继续执行、局部修补、后缀重规划或停止是否更合适。

本文并不求解完整 POMDP。完整 POMDP 需要明确状态空间、动作空间、转移概率、观测概率和奖励函数，而蛋白质设计 workflow 中的工具组合、失败原因、目标偏好和成本约束难以完全形式化。为适应工程实现，本文采用 Lite belief-state / 轻量信念状态近似表示运行时状态，将 evidence sufficiency、risk level、budget pressure 和 recovery need 等特征组织为可记录、可解释的状态摘要。这样既保留了 POMDP 中“在部分观测下依据信念行动”的思想，也避免了完整概率模型难以落地的问题。

1.3.3 大语言模型 Agent 协作

大语言模型 Agent 是近年来人工智能应用中的重要方向。与只输出文本的语言模型相比，Agent 更强调在环境中执行任务、调用工具、接收反馈并调整后续行动。ReAct 将推理和行动结合，使模型能够在中间观察的反馈下逐步处理复杂任务[8]。Toolformer 从工具调用角度说明，语言模型可以学习在适当位置调用外部工具，以补足计算、检索和外部交互能力[9]。这些方法可以为 LLM Agent 参与蛋白质设计 workflow 提供直接参照。

科研工作流中的 Agent 协作不能完全依赖语言模型的生成能力。蛋白质设计任务涉及工具调用、成本控制、安全边界和结果追踪，如果某一步判断失误就可能让后续结果不可用或不易解释。因此，系统需要把组件职责划分清楚。本文将系统划分为 PlannerAgent、ExecutorAgent、SafetyAgent 和

SummarizerAgent 等角色：PlannerAgent 处理目标理解和候选工作流生成，ExecutorAgent 按工具契约执行步骤，SafetyAgent 识别风险并触发人工确认，SummarizerAgent 汇总结果和报告。这样的角色分离可以避免单一 Agent 同时承担规划、执行、安全审查和总结任务而造成职责混杂。

在本文框架下，MDP/POMDP 理论与 LLM Agent 机制并不是两条彼此割裂的线索。MDP/POMDP 主要用于理解工作流中的动态决策和部分可观测性，LLM Agent 则承担起自然语言目标解析、候选计划生成和结果解释。CEBRA-WP 正是在两者之间建立起工程连接：它不把大模型输出直接当成最终执行计划，而是通过约束过滤、证据评分、轻量信念状态更新和恢复动作选择，把 Agent 生成的候选工作流转化为可执行、可审计、可恢复的系统行为。

1.4 本文的主要研究内容

本文研究对象不是新的蛋白质生成模型，而是面向蛋白质设计任务的多 Agent 协作系统、工作流规划算法和工程实现。系统将已有工具作为可调用能力，结合自然语言目标、结构化约束、ProteinToolKG、FSM、HITL 和运行时恢复机制，完成任务接入、候选生成、执行、证据记录和报告输出。

本文主要研究内容可以概括如下。

- 1) 构建统一任务和工具表示。本文将用户目标、约束、工具能力、输入输出字段、成本与安全等级纳入结构化契约，并用 ProteinToolKG 描述工具之间的组合关系，从而为候选生成、合法性检查、工具调用和恢复提供重要基础。
- 2) 设计多 Agent 协作架构。系统按输入交互、智能规划、工作流执行、安全汇总和资源管理组织功能，分离 PlannerAgent、ExecutorAgent、SafetyAgent 和 SummarizerAgent 职责；同时用 FSM、PendingAction/Decision、事件日志和任务快照约束任务生命周期。
- 3) 提出约束与证据感知、信念引导、恢复自适应工作流规划（Constraint-

and Evidence-aware Belief-guided Recovery-adaptive Workflow Planning, CEBRA-WP) 算法。该算法以目标 g 、约束集合 C 、ProteinToolKG K 、历史 h_t 、观测 o_t 和 Lite belief-state / 轻量信念状态 x_t 为输入，经过候选生成、硬可行性过滤、静态评分、后验目标评分、运行时重排序和恢复动作选择，输出候选集合、默认建议和恢复动作。算法支持 `continue`、`patchlocal`、`suffixplan` 和 `stop` 四类动作，并将 `stop` 映射为受 FSM 和 HITL 约束的终止型重规划候选。

- 4) 实现系统原型并开展测试与实验验证。后端采用 Python、FastAPI 和 Pydantic 实现任务接入、计划生成、工作流执行、HITL 接口、事件查询和报告读取；前端采用 React、TypeScript 和 Vite 构建轻量 Web 工作台；工具层通过统一适配器封装 ProtGPT2、ProteinMPNN、ESMFold/OpenFold、Biopython QC、DSSP 和目标评分等能力。本文通过 API 合约、数据契约、FSM 迁移、HITL 决策边界、快照恢复、前端与 CLI 可用性、安全阻断和恢复路径等测试验证系统可用性，并通过 `static_top1`、`fixed_threshold_gate`、`dynamic_no_belief_state` 和 `lite_belief_state` 四组策略对照实验分析 CEBRA-WP 的机制作用。

后文依次介绍相关技术、需求分析、系统设计、系统实现、测试验证、实验结果，以及总结与展望。

第2章 相关技术与理论基础

2.1 蛋白质计算设计基础

蛋白质由氨基酸序列折叠形成结构，并进一步影响结合、催化、稳定性等功能。蛋白质设计是在给定结构、功能或属性约束下寻找合适序列；*de novo* 设计则强调从目标条件生成新的候选序列或结构。对本文而言，该过程还必须可拆分、可组合、可恢复。

这一问题在特性上呈现出明显的高维组合搜索特征。对于一个长度为 L 的蛋白质序列来说，其理论上的搜索空间规模能够达到 20^L 这一量级。即便在实际的搜索过程当中，由于受到物理规律、生物进化过程以及具体任务要求的各种约束限制，其所形成的候选空间规模也依旧远远超出了人工进行枚举操作的能力范围。针对这一情况，现代蛋白质设计的工作流程通常会把生成、预测、筛选以及评分等环节拆解为多个不同的阶段，并借助一整套工具链来协作完成对候选范围的逐步收窄工作。

本文采取六阶段能力分层理解 *de novo* 设计流程：序列探索用于生成候选序列；结构映射用于预测候选序列可能折叠出的三维结构；质量门禁用于剔除长度、残基合法性、低复杂度或结构完整性不满足要求的候选；结构条件精修依据结构反馈重新设计序列；目标评分综合稳定性、结构质量和任务目标匹配度对候选排序；结果汇总则生成面向科研人员对报告。

这六个阶段是可组合的能力单元，实际任务可能跳过部分阶段，也可能在结构预测与结构条件精修之间迭代。第四章将基于这一能力分层设计系统 workflow。

2.2 蛋白质设计与分析工具

深度学习模型和生物信息学工具是本文系统的底层能力来源之一。本节从系统集成角度对这些工具进行归类，重点说明了它们在工作流中的作用。

AlphaFold2 在很大程度上提升了蛋白质结构预测的精确程度，目前正在

结构预测研究领域被视作一项非常关键的基准参考^[2]。随后的 AlphaFold3 则把研究范围进一步扩展到针对复杂生物分子之间相互作用的结构预测工作当中^[3]。ESMFold 则利用蛋白质语言模型表征进行快速结构预测，在速度和部署便利性方面具有一定优势^[6]。

在涉及序列设计与生成的环节里，ProteinMPNN 能够根据已有的蛋白质骨架结构去反向推导出相应的氨基酸序列，这种方式适合带有结构条件约束的逆向折叠任务^[5]。ProtGPT2 则是选用蛋白质语言模型的形式来生成类天然的序列，这种方法可以被用来进行候选序列的初步探索工作^[4]。另外，RFdiffusion 借助扩散生成这一视角有力地推动了 de novo 结构与功能方面的设计工作，从而为生成式蛋白质设计这一领域打下了厚实的技术背景基础^[7]。

传统平台和重要基础库同样不能忽略。Rosetta3 长期用于支持基于物理能量函数的蛋白质建模与设计，是传统蛋白质设计平台的代表^[17]。Biopython 则为序列解析、结构文件处理和重要基础生物信息操作提供了较稳定的接口^[18]。

这些工具各自侧重点不同，但单个工具并不能自动解决跨工具编排问题。结构预测工具关注预测精度，序列设计工具关注候选生成，重要基础库更侧重数据处理。本文系统通过 ToolAdapter 和 ProteinToolKG 把这些能力组织为可组合工具链，并在运行时根据证据、成本和风险选择后续动作。

2.3 科学工作流与可复现执行

科学工作流管理系统（SWMS）主要用于组织计算流程、管理依赖和记录执行过程。Pegasus 强调抽象工作流到分布式资源的映射^[19]，Nextflow 支持 DSL（领域特定语言）、容器化、缓存和可复现运行^[11]。可以看到，以上这些系统所具备的技术能力，都能够在很大程度上为本文所设计的蛋白质设计工作流提供重要的方法论参考。

通用工作流系统擅长执行预定义 DAG，但本文还关心运行中是否应改变计划。若步骤成功但质量不足，或高代价工具失败，系统需要判断是否保留已完成前缀、替换未执行后缀，而不是只记录完成、重试或重新调度。

因此，本文没有把全局控制流交给外部引擎，而是使用自定义 Workflow/FSM 管理任务生命周期。外部工具或流程后端可作为单个 PlanStep 的执行方式接入，但任务创建、候选确认、运行时恢复、人在环决策和审计链路由系统自身控制。该选择并不否定科学工作流的价值，而是将其定位于执行资源层或单步工具后端，将语义级恢复控制保留在本文系统中。

2.4 大语言模型工具调用

大语言模型（Large Language Model, LLM）可以用于自然语言理解、任务分解和工具使用。针对如何更有效地利用这些模型，学术界已经提出了一系列具有代表性的方法论，例如 ReAct、Toolformer、Tree of Thoughts 和 Reflexion 分别从推理行动、工具调用、多候选搜索和失败反馈角度提供了方法参考 [8,9,20,21]。本文系统中的 LLM Agent 主要承担起任务入口、候选生成和解释。

但蛋白质设计不同于一般文本任务：工具调用有真实成本，输出必须满足 schema，安全和预算不能只靠自然语言判断，任务状态也要在重启和等待人工确认时保持一致。OSWorld 等工作同样提示，Agent 的执行、恢复和稳健性需要被显式评估^[22]。

本文系统采用多 Agent 职责划分：PlannerAgent 生成候选工具链和解释，ExecutorAgent 调用已确认的工具步骤，SafetyAgent 提供风险信号，SummarizerAgent 汇总最终结果。LLM 可参与候选生成和解释，但候选硬性过滤、FSM 状态转移、HITL 决策应用和运行时恢复动作不由 LLM 完全掌控，而是由结构化算法和系统契约控制。这样可以把 LLM 能力嵌入到工作流，而不是让 LLM 独占控制权。

2.5 部分可观测规划与预算感知决策

MDP 和 POMDP 为运行时恢复提供理论参照。本文不求解完整 POMDP，而只借鉴部分可观测决策思想：系统能看到步骤结果、错误类型、质量指标、安全信号和预算消耗，却不能直接知道当前工具链是否仍有全局成功可能。因

此，瞬时观测不足以支撑可靠恢复决策。

部分可观测规划研究认为，在环境状态不能被完全观测时，决策者需要维护 **belief-state**，以历史观测的摘要形式估计当前状态^[16]。后续部分可观测启发式规划研究进一步强调，启发式函数应考虑随机效应与信息价值^[23]。本文把这一思想工程化为 **Lite belief-state** / 轻量信念状态：使用 p_{succ} 、 p_{sf} 、 r_{rec} 、 c_{rem} 、 e_{suf} 五个变量刻画运行时状态。

预算约束同样是高代价科研工作流的重要因素。预算强化学习研究说明，资源消耗和风险约束应被纳入决策目标，而不能只在任务结束后再统计^[24]。在本文场景中，这一思想在本文中体现为 **CEBRA-WP** 对 c_{rem} 、 b_{press} 和高代价调用的显式建模。证据不足时，系统会降低高成本候选的排序，在局部失败后先判断能否通过 **patchlocal**（局部修补）保留有效前缀，在恢复余量不足时生成受控的 **stop**（终止型重规划候选）。

2.6 相关系统与本文定位

近期蛋白质设计相关预印本已经开始关注多目标搜索、属性引导和系统化设计流程。例如，**RosettaSearch** 研究蛋白质序列设计中的推理时多目标搜索^[1]，**AutoBinder Agent** 探索面向 **binder** 设计的端到端 **Agent** 系统^[23]，**property-driven inverse folding** 相关工作则讨论多目标偏好和 **developability** 约束^[24]。这些工作从不同角度说明，蛋白质设计正在由单模型预测转向多目标、多工具和系统化流程。

本文不提出新的蛋白质生成模型，也不宣称湿实验验证，而是关注 workflow 控制：如何组织已有工具，如何用 **ProteinToolKG** 约束工具链，如何用 **Lite belief-state** / 轻量信念状态进行运行时重排序，以及如何在失败、风险和预算压力下保留可审计恢复链路。

此外，**ProteinGuide** 和 **ProteinZero** 等预印本从属性引导、在线反馈和自我改进角度讨论蛋白质生成模型的改进方向^[27,28]。本文与这些工作不同，重点放在工具编排、状态控制、人在环决策和恢复审计等工作流层问题。

图 2-1 给出了本文技术路线涉及的主要技术关系。

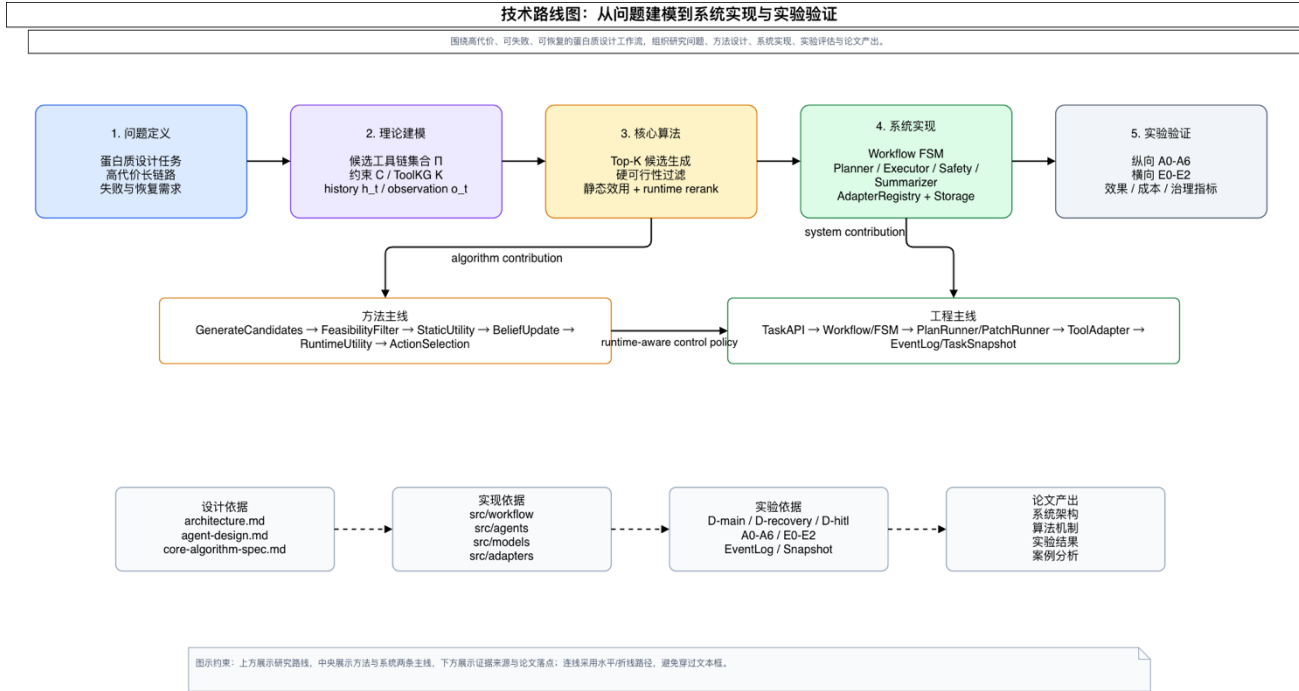


图 2-1 技术路线概述

图 2-1 概括展示了本文系统与相关技术方向的关系：蛋白质设计工具提供底层计算能力，科学工作流提供执行与复现方面的重要基础，LLM Agent 提供候选生成与解释能力，部分可观测规划和预算感知决策则为 CEBRA-WP 的运行控制提供理论依据。

2.7 本章小结

本章界定了蛋白质工具、科学工作流、LLM Agent、部分可观测规划和预算感知决策等基础概念。它们共同说明，本文系统需求不是单一工具能力，而是跨工具编排、运行时控制和证据追踪。

第3章 需求分析

本章从蛋白质设计工作流的实际问题出发，归纳系统功能需求、非功能需求和边界约束。与单步工具封装不同，本文系统面向高代价、长链路、可失败、可恢复的科研工作流，因此需覆盖候选计划、HITL、运行时自适应和审计追踪。

3.1 问题界定

本文处理的是多工具、多阶段、多状态的工作流控制问题，而非底层模型训练。蛋白质设计通常由目标描述、候选生成、结构预测、质量筛选、目标评分和结果汇总组成；各工具在接口、成本、失败模式和适用边界上差异明显。

固定流水线难以根据中间证据调整计划，也不便在高代价失败后保留有效前缀。科学工作流系统能支持流程定义和可复现执行^[11,19]，但本文还需处理质量信号、失败语义、预算压力和人工确认。开放环境 Agent 评测也说明，应显式评估执行、恢复和稳健性^[22]。

固定流水线与本文系统方案的差异如图 3-1 所示。

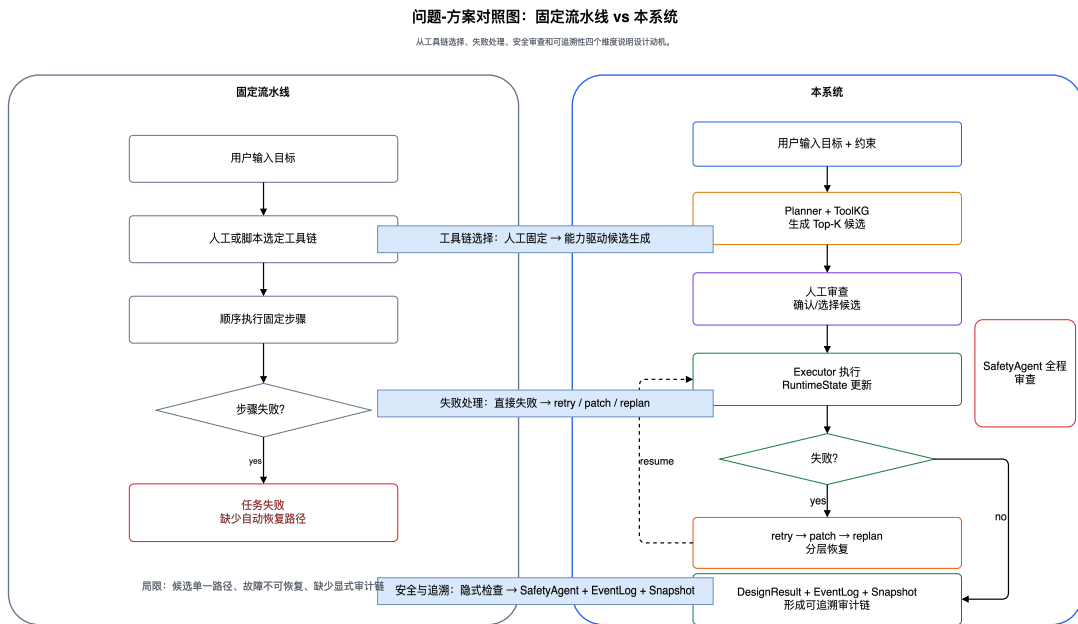


图 3-1 固定流水线与本文系统的问题-方案对照

图 3-1 左侧描述的是固定流水线模式：任务开始前由人工或脚本选定工具链，执行过程中一旦步骤失败，往往只能重跑或结束任务。右侧描述的是本文系统方案：用户目标和约束进入系统后，PlannerAgent 与 ProteinToolKG 生成候选工具链，经可行性过滤和人工确认后交由 ExecutorAgent 执行；执行中若出现失败、风险或证据不足，系统按 retry、patch_local、suffix_replan 和 terminal_stop 的层级路径进行恢复。图中还标出 SafetyAgent 的风险信号，以及 EventLog、TaskSnapshot 形成的审计链，这些机制一起构成后续系统设计的需求来源。

3.2 用户角色与使用场景

系统主要面向两类使用者。

第一类是提交蛋白质设计任务并审查关键决策的科研人员。他们关注任务目标是否被正确解析，候选工具链是否合理，高成本步骤是否值得执行，失败后的修补或重规划是否保留已有有效结果，以及最终报告是否包含必要的序列、结构、指标、风险和审计信息。

第二类是系统开发与实验人员。他们关注工具接入是否规范，工作流失败是否可定位，不同 runtime policy 是否能够复现实验，日志、快照、候选分数、动作效用和实验指标是否完整。

围绕上述用户，系统应支持以下典型场景。

- （1） 用户提交自然语言设计目标，并补充长度范围、安全等级、结构模板、预算等约束；系统将其转化为结构化任务。
- （2） 系统根据任务目标和 ProteinToolKG 生成 Top-K 候选计划，并展示默认建议、风险等级、成本估计、解释依据和可能的工具链差异。
- （3） 当初始计划包含高代价或高风险步骤时，系统进入人工确认状态，等待用户接受、拒绝或选择候选。
- （4） 执行过程中若出现工具失败、schema 不满足、质量门禁失败、结构预测置信度不足或安全警告，系统根据失败上下文生成局部修补或后

缀重规划候选。

- (5) 任务完成后，系统生成包含序列、结构文件、指标、风险标记和恢复历史的报告，并保留事件日志、快照、决策记录和中间产物路径。

3.3 功能需求

3.3.1 任务接入与约束确认

系统应提供统一的任务接入能力，将自由文本目标、结构化约束和任务元数据收敛为 **ProteinDesignTask**。蛋白质设计任务通常存在信息不完整的情况，因此任务接入不应只是单次表单提交，而应支持草稿创建、字段补充、场景门控和正式确认。

该模块应满足以下需求。

- (1) 为每个任务生成唯一 **task_id**，记录目标、约束、创建时间和元数据。
- (2) 支持自然语言目标、兼容自由文本入口和已确认结构化任务规格，并保证正式任务创建入口语义互斥。
- (3) 在正式执行前进行场景可行性和工具 **readiness** 预览，避免明显不可执行任务直接进入高代价工作流。
- (4) 保留任务草稿、字段补充和确认过程，使后续计划生成能够追溯用户目标与系统解析之间的关系。

3.3.2 候选计划生成与工具链选择

系统应根据任务目标、约束集合和 **ProteinToolKG** 中的工具能力信息生成候选计划。候选计划不应只包含工具名称，还应包含步骤依赖、输入引用、输出字段、风险等级、成本估计、评分分解和解释说明。计划的最小执行单元是 **PlanStep**，多个步骤通过 **S{id}.{field}** 形式表达中间结果引用。

候选生成必须先做硬可行性过滤：工具存在，输入输出闭合，**schema** 合法，安全等级和硬预算不被突破。对证据不足或能力降级但未触犯硬约束的候选，可保留为 **degraded feasible**，并要求人工确认或策略授权。

3.3.3 workflow执行与工具适配

系统需要能够按照已确认的 Plan 调用具体工具，并将每一步执行结果收敛为统一的 StepResult。工具调用通过 ToolAdapter 完成，ExecutorAgent 只依赖适配器注册表和抽象接口，不直接绑定具体工具内部实现。

执行模块应满足以下需求。

- （1） 解析步骤输入，包括常量输入和上游步骤输出引用。
- （2） 统一记录工具身份、适配器身份、执行模式、远程作业 ID、错误类型、指标和产物路径。
- （3） 对步骤失败进行分类，区分可重试错误、不可重试错误、工具错误、安全阻断和质量门禁失败。
- （4） 在局部失败时优先遵循 retry → patch → replan 的恢复顺序。
- （5） 允许外部流程后端作为单个 PlanStep 的执行方式接入，但多步控制流仍由 Workflow/FSM/Plan 负责。

3.3.4 HITL 与决策审查

蛋白质设计工作流中存在高风险、高成本和高不确定性的节点，系统不能把所有关键选择都交给自动逻辑。因此，系统需要显式支持 HITL（人在环决策）。在本文系统中，HITL 不是 Agent 临时等待自然语言输入，而是通过 PendingAction 和 Decision 两个结构化对象完成。

系统需支持初始计划、局部修补和后缀重规划三类人工确认。进入 WAITING_* 状态前，必须持久化候选、默认建议、解释和快照；Decision 生效后再应用对应 Plan、PlanPatch 或 Replan。算法建议的 stop 以 terminal_stop 候选呈现，接受后进入 FAILED，主动取消则为 CANCELLED。

3.3.5 运行时自适应与恢复决策

固定流程难以充分利用执行过程中的失败码、质量指标、预算消耗和安全信号。因此，系统需要维护 Lite belief-state / 轻量信念状态，并在实现中通过

`RuntimeState` 持久化，用于描述当前任务继续执行的成功概率、结构性失败概率、恢复余量、预期剩余成本和证据充分性。

CEBRA-WP 需支持 `continue`、`patch_local`、`suffix_replan` 和 `stop` 四类动作。动作选择不能只看单步成败，还结合执行阶段、失败类型、重试耗尽、安全阻断、运行时状态和候选效用；它的目标是在高代价步骤前减少无效调用，在失败后先判断能否保留前缀并局部修补。

3.3.6 结果汇总、报告与审计

系统最终输出不应仅是单一序列或结构文件，还需要包含任务目标、计划版本、步骤结果、结构与质量指标、安全事件、恢复轨迹和报告路径。任务进入汇总阶段后，`SummarizerAgent` 负责生成便于用户后续阅读的 `DesignResult` 和报告文本。

同时，系统还要保留完整审计链。事件日志记录状态变化、候选生成、人工决策、工具执行和恢复动作；任务快照保存执行上下文、已完成步骤、运行时状态和待决策对象。对于长时间运行任务和远程模型调用，这类记录也是可靠性分析和实验复现的重要基础。

3.4 非功能需求

3.4.1 可追溯性

任何计划生成、候选推荐、人工确认、状态迁移和恢复动作都应能从日志或快照中追踪其依据。候选对象中的 `source_refs`、`score_breakdown`、`runtime_adjustment`、`action_utility` 和 `runtime_state_summary` 等字段均应服务于解释与复核。

3.4.2 可扩展性

新的蛋白质设计或分析工具应通过 `ToolAdapter` 接入，并在 `ProteinToolKG` 中声明能力、输入输出、成本、风险和可用性信息。`ExecutorAgent` 不应因新增具体工具而改变核心调度逻辑。`PlannerAgent` 应基于工具能力和约

束生成候选，而不是把某个工具链写死在代码中。

3.4.3 可靠性与可恢复性

任务状态迁移必须受 FSM 约束。进入等待状态前必须完成 PendingAction 和快照写入。系统重启后应能够根据最新快照和事件日志恢复执行上下文；恢复到等待态后不应自动推进，以避免绕过人工确认。

3.4.4 安全与边界控制

SafetyAgent 负责输入、步骤和输出阶段的风险判定，但不直接决定终态或重规划。安全信号应作为 Planner、Workflow 和 HITL 的触发依据。系统不得将计算验证结果扩大为湿实验结论，也不得将未验证指标写成已完成生物学验证。

3.4.5 实验可复现性

论文实验需要比较 static_top1 、 fixed_threshold_gate 、 dynamic_no_belief_state 和 lite_belief_state 等策略组，因此系统应稳定记录 runtime policy、候选得分、动作 utility、高代价调用次数、恢复成功情况、最终任务状态和运行产物路径。

3.4.6 需求优先级与边界

从论文论证和系统实现两个角度看，需求优先级可以分成三层。

第一层是必须满足的控制闭环需求，涵盖任务接入、候选计划生成、FSM 状态控制、工具执行、HITL、事件日志和结果汇总。系统能否端到端运行，首先取决于这些能力是否成立。

第二层是算法机制需求，主要包含硬可行性过滤、RuntimeState 更新、运行时的重排序、patch_local/suffix_replan 候选生成和 stop 候选语义。这些能力会决定 CEBRA-WP 是否能在系统中落地并接受验证。

第三层是扩展性需求，包括更多工具接入、更复杂的结构可视化、更大规模任务矩阵和更丰富的外部基线。它们属于后续演进方向，不应被表述为本文

已经全部完成的能力。

本文边界同样需要说明清楚：系统不重新实现 AlphaFold、ESMFold、ProteinMPNN 等底层模型，也不声称自动验证候选蛋白的真实生物功能，也不把机制验证扩大为生产级可靠性证明。本文重点关注工作流层的规划、执行、恢复和审计。

3.5 本章小结

本章归纳了任务接入、候选计划、工具执行、HITL、运行时恢复和结果审计六类功能需求，并提出可追溯性、可扩展性、可靠性、安全边界和实验可复现性等非功能要求。

上述需求共同指向一个设计方向：以统一任务契约作为入口，以 FSM 作为控制骨架，以多 Agent 划分职责，以 ToolAdapter 和 ProteinToolKG 承载工具能力，并由 CEBRA-WP 负责运行时规划与恢复控制。下一章将在这些约束下展开系统设计。

第4章 系统设计

本章在需求分析的基础上说明系统总体设计。前一章已经将课题目标细化为任务接入、候选计划生成、工具执行、HITL、运行时恢复和审计追踪等需求。基于这些需求，本文系统设计的重点不是把若干工具串接成固定流水线，而是在可执行约束、安全约束和预算约束下，为蛋白质设计任务提供可规划、可恢复、可审计的工作流运行框架。

本章先介绍系统分层架构、核心组件、有限状态机和六阶段蛋白质设计工作流，再定义约束与证据感知、信念引导、恢复自适应工作流规划（Constraint-and Evidence-aware Belief-guided Recovery-adaptive Workflow Planning , CEBRA-WP）。针对 CEBRA-WP，本章给出其在本文系统中的形式化对象、评分函数、运行时状态、恢复动作和实验可验证性。

4.1 设计目标与总体架构

蛋白质设计工作流不同于一般 Web 后端任务：自然语言目标需要转为可执行步骤，结构预测和重型打分成本较高，中间失败也不必然导致任务整体失败。参数错误、输入引用缺失、质量门禁未通过或外部工具暂不可用，都可能通过局部修补或后缀重规划恢复；系统输出还需保留候选依据、人工确认、执行结果和恢复历史。

基于这些特点，系统架构采用五层划分，如图 4-1 所示。

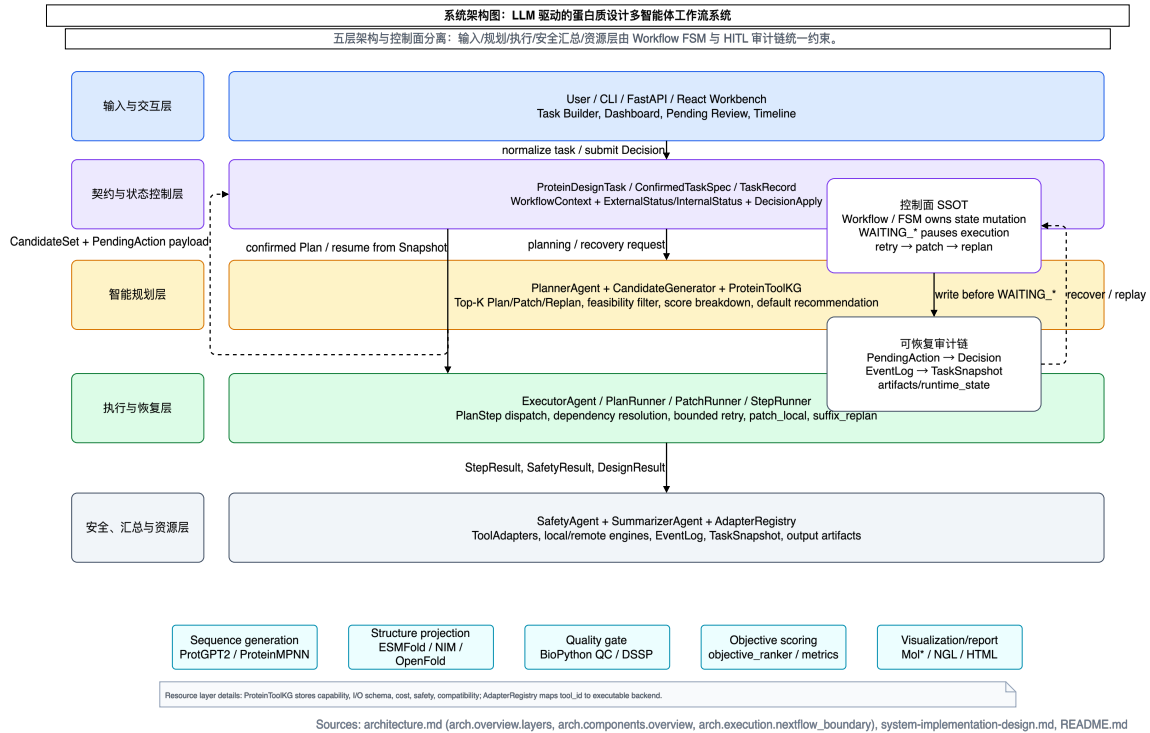


图 4-1 系统五层分层架构

输入层面向 Web 工作台、CLI 和 API，接收用户的自然语言目标、结构化约束和人工决策。智能规划层以 PlannerAgent、ProteinToolKG 和 CEBRA-WP 策略为核心，承担候选计划生成工作、可行性过滤、评分和恢复候选生成。执行层由 ExecutorAgent、PlanRunner 和 StepRunner 组成，按照已确认的计划调用工具并记录步骤结果。安全与汇总层包含 SafetyAgent 和 SummarizerAgent，分别处理风险判定和最终报告生成。资源层则包括 ToolAdapter 注册表、ProteinToolKG、事件日志、任务快照和文件产物管理。

五层之间依靠结构化数据契约交换信息。输入层不直接执行工具，规划层不直接修改任务终态，执行层不越过有限状态机跳转状态，安全层不直接编辑计划，汇总层也不重新执行计算。经过这样的职责拆分，候选生成、工具调用、人工确认和恢复动作都可以回到相应的契约对象与事件记录中，复杂任务的可追溯性因此有了实现基础。

4.2 核心组件与职责边界

系统核心逻辑主要由四类 Agent 和工具适配层共同完成。

PlannerAgent 承担计划生成和恢复候选生成。它读取任务目标、约束、执行历史和 ProteinToolKG，生成初始 PlanCandidate，也会在失败后生成局部修补候选与后缀重规划候选。PlannerAgent 的输出必须是结构化候选，而不是难以验证的自然语言建议。每个候选包含候选标识、摘要、结构化载荷、评分分解、风险等级、成本估计、解释文本和来源引用，因此既能被自动策略排序，也能呈现在人工审查界面中。

ExecutorAgent 主要负责工具调用和计划推进。它通过 PlanRunner 管理计划级执行流程，通过 StepRunner 处理单个步骤的输入解析、上游引用求解、适配器调用和结果写入。ExecutorAgent 能够识别失败、重试耗尽和安全阻断等信号，但不会直接绕过 Planner 生成修补方案，也不会在等待人工确认时继续执行工具。

SafetyAgent 是风险信号源。它在输入、执行过程和输出阶段产生安全判定，输出 ok、warn、block 等等级。warn 可触发人工确认，block 可阻断自动推进并触发重规划候选。SafetyAgent 的定位是风险判定与建议，不负责计划搜索和工具执行。

SummarizerAgent 主要负责结果汇总。它读取计划、步骤结果、安全事件和恢复历史，生成面向用户的报告和机器可读的 DesignResult。该组件的输出只反映已经完成的计算和已有证据，不把未验证推断写成实验结论。

ToolAdapter 层为外部工具提供统一调用接口。工具注册表面向执行层，回答“如何调用工具”；ProteinToolKG 面向规划层，回答“哪些工具可用于何种能力、输入输出如何兼容、成本和风险如何估计”。二者共同支撑候选生成与执行验证。ProteinToolKG 的局部结构如图 4-2 所示。

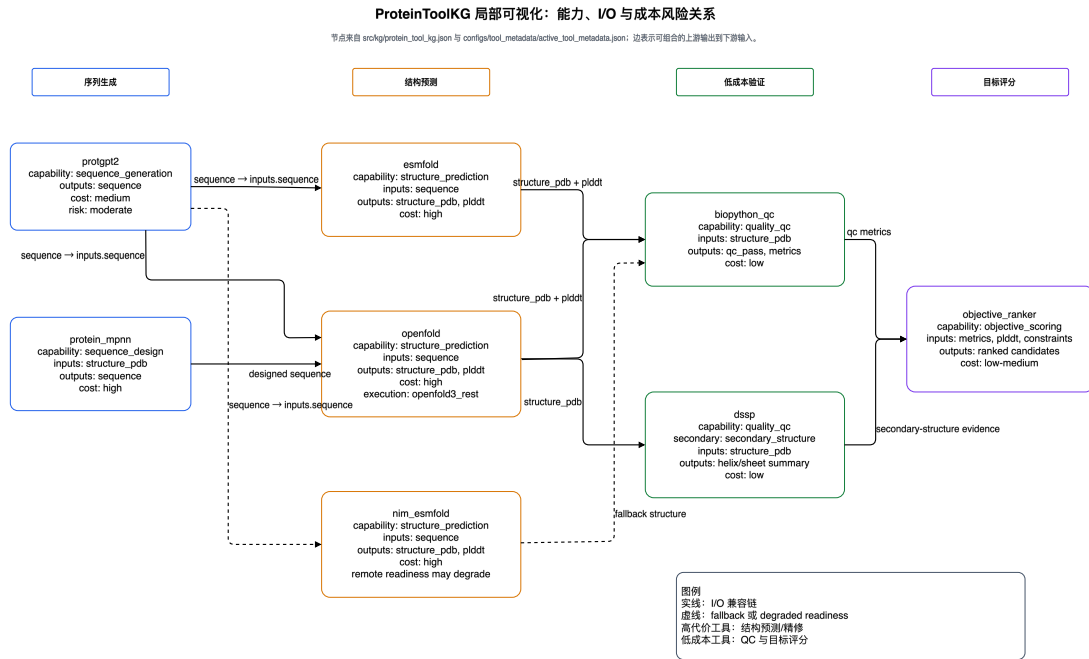


图 4-2 ProteinToolKG 局部可视化

图 4-2 展示了 ProtGPT2、ProteinMPNN、ESMFold/OpenFold、Biopython QC、DSSP 和 objective_ranker 等代表性工具之间的能力节点、输入输出字段、成本风险属性和 I/O 兼容边。这里需要说明的是，本文使用的 ProteinToolKG 是轻量工具能力索引，并不是完整的生物知识图谱。它主要服务于系统中的 Planner 的能力发现、约束校验和候选解释。

4.3 任务生命周期与 HITL 约束

系统使用有限状态机（Finite State Machine, FSM）控制任务生命周期，如图 4-3 所示。

任务对外状态包括 `CREATED`、`PLANNING`、`WAITING_PLAN_CONFIRM`、`PLANNED`、`RUNNING`、`WAITING_PATCH_CONFIRM`、`WAITING_REPLAN_CONFIRM`、`SUMMARIZING`、`DONE`、`FAILED` 和 `CANCELLED`。`WAITING_*` 状态具有明确语义：系统已经暂停自动推进，等待人类提交结构化 `Decision`。`DONE`、`FAILED` 和 `CANCELLED` 为终态，进入后不再被自动修改。

24

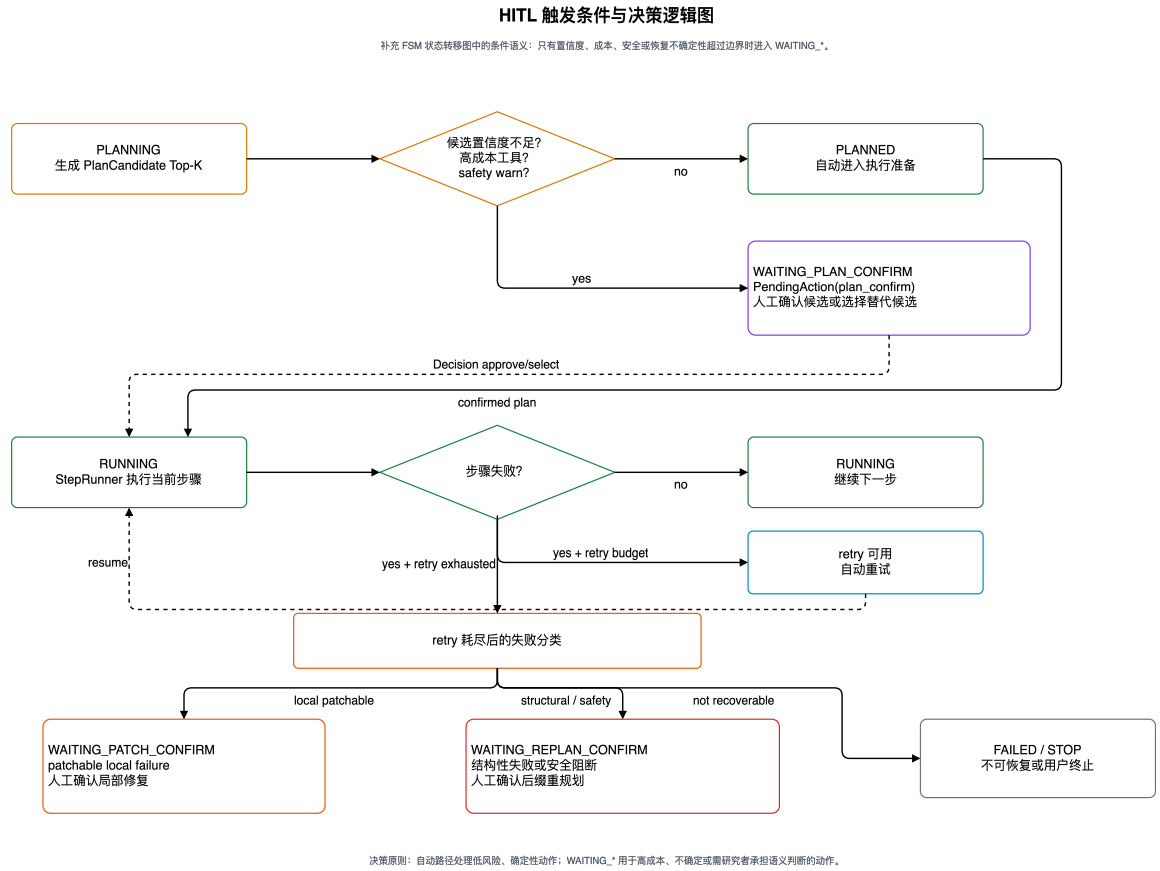


图 4-4 HITL 决策条件

PLANNING 阶段，如果候选置信度不足、即将调用高代价工具或 SafetyAgent 给出 warn，系统进入 WAITING_PLAN_CONFIRM。RUNNING 阶段，如果步骤失败但仍有 retry budget，系统先进行有界重试；若重试耗尽且局部可修复，则进入 WAITING_PATCH_CONFIRM；若出现结构性失败、安全阻断或恢复余量不足，则进入 WAITING_REPLAN_CONFIRM。人工决策通过 Decision 契约提交，Decision Apply 模块验证 pending action 与候选绑定关系，再推动 FSM 合法迁移。

这一设计为 CEBRA-WP 提供了较清晰的控制边界：算法可以生成候选、计算分数和提出动作建议，但任务状态仍由 FSM 统一推进；算法建议不能绕过 WAITING_* 状态，也不能把 stop 隐式处理为不可审计的失败。

4.4 六阶段蛋白质设计 workflow

蛋白质设计能力被组织为六个阶段，如图 4-5 所示。

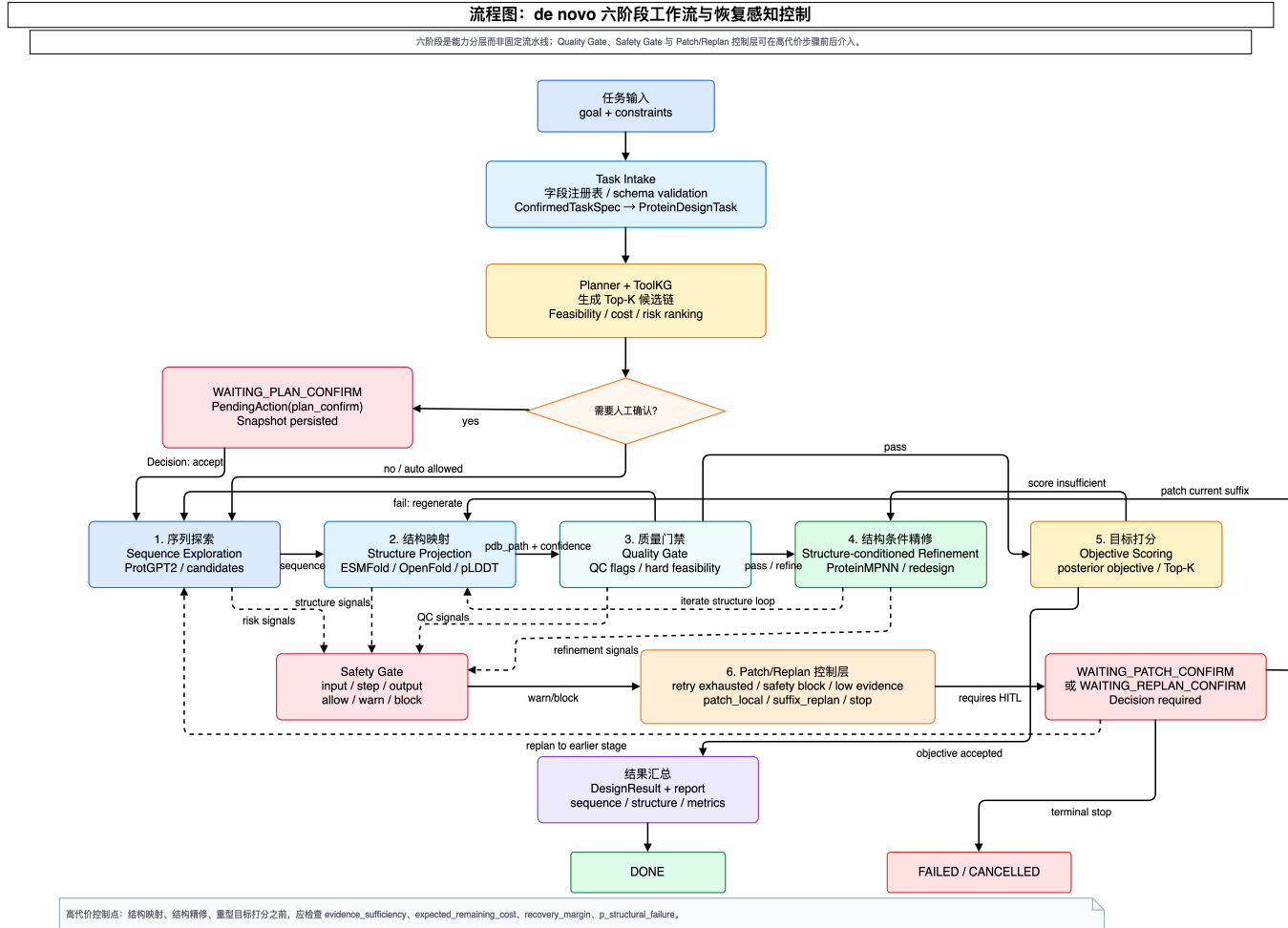


图 4-5 六阶段 de novo 蛋白质设计 workflow

六个阶段可以依次理解为序列探索、结构映射、质量门禁、结构条件精修、目标评分和结果汇总。序列探索负责产生候选序列；结构映射把序列转换为结构并产生折叠置信度；质量门禁检查序列合法性、结构完整性和低复杂度等工程质量；结构条件精修依据结构反馈开展重设计；目标评分对候选进行多目标排序；结果汇总生成 DesignResult 和报告。

六阶段是能力上的分层，而不是固定流水线。质量门禁失败可以回到序列

探索，目标评分不足可以回到结构条件精修。CEBRA-WP 的作用是在可替代路径间进行受约束的选择：证据不足时延后高代价步骤，局部失败可修复时优先 `patch_local`，结构性失败时转向 `suffix_replan` 或 `stop`。

4.5 CEBRA-WP 的提出背景

上一节给出了蛋白质设计工作流的能力分层和可替代路径，但阶段划分本身并不能回答何时继续执行、何时局部修补、何时后缀重规划以及何时终止止损。CEBRA-WP 的提出，正是为了处理这些运行时决策。

固定流水线适合步骤稳定、失败语义简单的任务，但蛋白质设计的关键风险具有部分可观测性。结构预测失败可能来自服务异常，也可能指向上游序列质量不足；质量门禁失败可能局部可修复，也可能意味着候选链路失效。因此，系统需要把运行时证据纳入规划。

CEBRA-WP 的设计动机主要来自三类研究：部分可观测规划强调 `belief-state`，预算约束研究要求将成本和风险作为重要决策变量^[16,25,26]，`Tree of Thoughts` 与 `Reflexion` 提供多候选搜索和失败反馈思路^[20,21]。本文将这些思想约束到可执行工具链规划中，使候选、恢复动作和人工确认均以结构化对象来表达。

基于这些考虑，CEBRA-WP 被定义为一种面向高代价科研工作流的约束化、证据感知、信念引导、恢复自适应规划算法。它不追求训练端到端最优控制器，而是以可解释、可配置、可审计的形式，把候选生成、硬约束过滤、静态效用、运行时状态、后验证据和恢复动作选择组织为一个闭环。

4.6 CEBRA-WP 形式化定义

CEBRA-WP 的完整名称为 `Constraint- and Evidence-aware Belief-guided Recovery-adaptive Workflow Planning`，本文译为“约束与证据感知、信念引导、恢复自适应的工作流规划”。当前论文版本为 `cebra_wp.v2`，其子公式包括 `static_score.v1`、`posterior_score.v1`、`runtime_adjustment.v1`、`action_utility.v1` 和

表 4-1 CEBRA-WP 主要符号说明

符号	含义	来源或载体	在算法中的作用
t	当前决策时间步	EventLog、 RuntimeEvaluator	索引一次候选生成、状态更新 和恢复动作选择
g	设计目标	用户输入 Con- firmedTaskSpec	定义任务目标与目标权重
C	约束集合	任务约束、策略配置	限定长度、安全、预算、 工具白名单和输出要求
K	ProteinToolKG	ProteinToolKG	提供工具能力、I/O schema、兼容关系、成本和风险
h_t	时间步 t 前的执行 历史	EventLog、TaskSnapshot	提供已完成步骤、 失败记录、恢复历史 和人工决策
o_t	当前运行时观测	StepResult、 SafetyResult、指标、错误 细节	更新运行时状态并计算后验证 数据
x_t	Lite belief-state / 轻量 信念状态	RuntimeState	表示成功概率、结构性失败概率、 恢复余量、剩余成本和证据充分度

表 4-1（续表）

符号	含义	来源或载体	在算法中的作用
$\mathbf{x}_{t+1}$	观测更新后的轻量信念状态	RuntimeState 更新结果	为候选重排和恢复动作选择提供更新后状态
$\Pi_{\text{raw},t}$	原始候选集合	PlannerAgent	包含初始计划、局部修补或重规划候选
Π_t	过滤后的候选集合	FeasibilityFilter	只保留可执行或受保护的 degraded feasible 候选
π	候选工作流	PlanCandidate、PatchCandidate、ReplanCandidate	作为过滤、评分和重排序的基本对象
π^*	默认候选	RuntimeEvaluator、Decision	作为自动执行或 HITL 审查的默认建议
S_{static}	静态效用	score_breakdown	在运行时观测介入前评价候选先验质量
G_{post}	后验目标匹配	posterior_objective	根据证据可靠性修正目标评分
Δ	运行时修正项	runtime_adjustment	根据状态变量修正候选排序
U_{π}	候选运行时效用	RuntimeEvaluator	输出 Top-K 和默认候选
$\mathbf{a}_t$	恢复动作	action_utility	在 continue、patch_local、suffix_replan、stop 之间选择

在时间步 t ，算法输入为 (g, C, K, h_t, o_t, x_t) ，输出为结构化 Decision 建议：候选集合 Π_t 、默认候选 π^* 、候选解释、运行时状态摘要和恢复动作 $\mathbf{a}_t$ 。核心计算过程可概括为：

$$\Pi_{\text{raw},t} = \text{GenerateCandidates}(g, C, K, h_t) \quad (4-1)$$

$$\Pi_t = \text{FeasibilityFilter}(\Pi_{\text{raw},t}, C, K, h_t) \quad (4-2)$$

$$S_{\text{static}} = \text{StaticUtility}(\pi, g, C, K) \quad (4-3)$$

$$x_{t+1} = \text{BeliefUpdate}(x_t, o_t, h_t) \quad (4-4)$$

$$G_{\text{post}} = \text{PosteriorObjective}(\pi, g, o_t) \quad (4-5)$$

$$U_\pi = \text{RuntimeCandidateUtility}(S_{\text{static}}, G_{\text{post}}, x_{t+1}) \quad (4-6)$$

$$a_t = \text{RecoveryAwareActionSelection}(x_{t+1}, \Pi_t, h_t, C) \quad (4-7)$$

该定义突出两点：一是候选生成与候选选择相互分离，Planner 可以生成多个方案，但只有通过可行性和效用评估的候选才会进入执行或人工确认；二是运行时状态只修正排序和动作建议，不改变 FSM、Agent 职责和硬约束。

4.7 候选过滤与静态效用

CEBRA-WP 支持三类候选。PlanCandidate 表示初始完整计划；PatchCandidate 表示对当前计划中局部步骤的参数级、工具级或结构级修补；ReplanCandidate 表示对未执行后缀或整体策略的替换，其中 suffix_replan 优先保留已验证前缀，terminal_stop 表示继续投入不划算时的终止型重规划候选。

候选进入评分前必须先通过硬可行性过滤。对候选 π 定义硬可行性谓词：

$$F_h(\pi, C, K, h_t) = F_{\text{tool}} \wedge F_{\text{schema}} \wedge F_{\text{io}} \wedge F_{\text{safety}} \wedge F_{\text{budget-hard}} \wedge F_{\text{availability}} \quad (4-8)$$

式中， F_h ——硬可行性谓词；

F_{tool} ——工具存在性约束；

F_{schema} ——输入输出 schema 约束；

F_{io} ——跨步骤 I/O 闭合约束；

F_{safety} ——安全约束；

$F_{\text{budget-hard}}$ ——硬预算约束；

$F_{\text{availability}}$ ——关键工具可用性或降级路径约束；

进一步说明则是， F_{tool} 检查候选中的工具是否存在于能力图或适配器注册表； F_{schema} 检查输入输出字段是否满足工具 schema； F_{io} 检查跨步骤引用是否闭合； F_{safety} 检查是否违反安全等级或触发 safety block； $F_{\text{budget-hard}}$ 检查是否突破不可逾越预算上限； $F_{\text{availability}}$ 检查关键工具是否可用或是否有明确降级路径。过滤后的集合为：

$$\Pi_t = \{\pi \in \Pi_{\text{raw},t} \mid F_h(\pi, C, K, h_t) = 1\} \quad (4-9)$$

硬可行性过滤承担系统边界约束，因此静态评分不能把硬不可行候选“打高分后救回”。工程实现可以保留 degraded feasible 候选用于解释或 HITL 审查，但这类候选必须携带降级原因和人工确认要求。

在候选通过硬过滤后，算法计算静态效用：

$$S_{\text{static}}(\pi) = w_f F_s(\pi) + w_g G(\pi; g, o_t) - w_c C_{\text{norm}}(\pi) - w_r R_{\text{norm}}(\pi) - w_{\text{rec}} \text{Rec}(\pi) + w_q Q(\pi) \quad (4-10)$$

式中， $F_s(\pi)$ ——候选 π 的软可行性分数；

$G(\pi; g, o_t)$ ——目标适配度；

$C_{\text{norm}}(\pi)$ ——归一化成本；

$R_{\text{norm}}(\pi)$ ——归一化风险；

$\text{Rec}(\pi)$ ——恢复复杂度；

$Q(\pi)$ ——工程可靠性项；

w_f, w_g, w_c ——目标、可行性和成本相关权重；

w_r, w_{rec}, w_q ——风险、恢复复杂度和工程可靠性相关权重；

F_s 表示候选在 schema 完整、工具 readiness、fallback depth 等方面的先验质量； G 是目标匹配度； C_{norm} 是归一化成本； R_{norm} 是归一化风险； Rec 是恢复复杂度； Q 是工程可靠性项。权重 w_* 由策略配置

给出，用于在不同任务设置下调节目标、成本、风险和恢复难度的相对重要性。

静态效用用于形成执行前的候选先验排序。它可以偏好成本更低、风险更小、恢复更容易的工具链，但无法感知执行中已经发生的失败、证据不足或预算压力变化。因此，静态效用之后还需要 Lite belief-state / 轻量信念状态和后验证据修正。

4.8 轻量信念状态与观测更新

CEBRA-WP 使用 Lite belief-state / 轻量信念状态近似表示执行过程中无法完全观测的工作流状态。它只保留三类变量：对恢复动作选择必要的变量、能从现有日志稳定更新的变量，以及可由实验和案例解释的变量。状态向量定义为：

$$\mathbf{x}_t = \begin{bmatrix} p_{succ} \\ p_{sf} \\ r_{rec} \\ c_{rem} \\ e_{suf} \end{bmatrix} \quad (4-11)$$

各状态量的语义如表 4-2 所示。

表 4-2 Lite belief-state / 轻量信念状态变量

状态量	取值范围	含义	主要更新来源	决策作用
p_{succ} 成功概率	[0,1]	当前链路继续执行 后完成任务的估计概率	步骤成功、质量指标、候选静态分、失败记录	支持 continue 与 stop 判断
p_{sf} 结构失败	[0,1]	当前链路遭遇结构性失败或后续必须重规划的估计概率	结构预测失败、质量门禁失败、安全阻断、重复失败	提高 patch_local、 suffix_replan、 stop 权重

表 4-2（续表）

状态量	取值范围	含义	主要更新来源	决策作用
r_{rec} 恢复余量	[0,1]	在保留有效前缀 前提下继续恢复 的余量	已完成步骤比 例、失败类型、 patch/replan 次数	区分局部修补与 后缀重规划
c_{rem} 剩余成本	非负实数	从当前状态到任 务结束的剩余成 本暴露	剩余步骤、工具 成本先验、预算 配置、重试记录	派生预算压力并 约束高代价动作
e_{suf} 证据充分	[0,1]	当前证据是否足 以支持进入更高 代价步骤	质量门禁、结构 指标、目标评 分、证据可靠性	控制高代价步骤 推进与人工确认

运行时观测 o_t 来自 StepResult、SafetyResult、失败上下文、局部修补/重规划历史、已完成步骤、剩余后缀和 HITL 决策记录。派生量如 b_{press} 、 v_{hitl} 、 l_{patch} 和 p_{pres} 不作为持久化主状态，而是根据当前状态和候选上下文按需计算。这样可以避免把与具体候选强相关的临时判断写入长期状态，从而降低状态漂移风险。

以预算压力为例， c_{rem} 保留剩余成本原始估计，不直接等同于 $[0,1]$ 区间内的预算压力。若任务给出预算上限 c_{cap} ，则：

$$b_{\text{press}} = \text{clip}\left(\frac{c_{\text{rem}}}{\max(c_{\text{cap}}, 0.1)}, 0, 1.5\right) \quad (4-12)$$

若任务未给出预算上限，则使用：

$$b_{\text{press}} = \text{clip}(c_{\text{rem}}, 0, 1.5) \quad (4-13)$$

式中， b_{press} ——预算压力；

c_{cap} ——任务预算上限；

$\text{clip}(\cdot)$ ——裁剪函数；

因此，预算压力是由任务上下文派生的动作决策量，而不是 RuntimeState

的持久化字段。该区分使系统既能记录可解释的剩余成本，也能在不同预算设置下统一比较恢复动作。

状态初始化可由首选候选的静态分数、风险分数、恢复复杂度、剩余成本先验和廉价证据覆盖率给出。例如， p_{succ} 可由静态候选分数裁剪得到， p_{sf} 可由风险项裁剪得到， r_{rec} 可由恢复复杂度的反向量估计， c_{rem} 来自剩余工具成本先验， e_{suf} 来自低成本质量证据覆盖率。随着执行推进，成功步骤会提升 p_{succ} 和证据充分度，结构性失败会提高 p_{sf} 并降低恢复余量，重复失败和预算消耗则会提高剩余成本压力。

该状态设计参考了部分可观测规划中的 **belief-state** 思想^[16]，但工程实现上保持轻量：系统不求解完整 POMDP，也不在线训练策略网络，而是借助一组可解释变量支撑候选重排和恢复动作选择。

4.9 后验目标评分与运行时重排序

蛋白质设计目标通常是多目标的，包括结构质量、稳定性、功能、**novelty** 和安全性等。不同目标的证据来源不同，证据可靠性也不同。为避免把弱证据与直接证据等价处理，CEBRA-WP 使用证据加权后验目标匹配：

$$G_{post}(\pi; g, o_t) = \sum_m \lambda_m(g) \rho_m(o_t) q_m(\pi, o_t) \quad (4-14)$$

这里， m 为目标维度， $\lambda_m(g)$ 是由任务目标 g 会决定的目标权重， $q_m(\pi, o_t)$ 是候选 π 在该维度上的归一化分数， $\rho_m(o_t)$ 是观测 o_t 对目标维度 m 的证据可靠性权重。证据状态分为 **direct**、**proxy**、**degraded** 和 **missing**：**direct evidence** 指已产生的结构质量指标，**proxy evidence** 指由轻量质量门禁推断出的间接信号，**degraded evidence** 表示工具降级或证据覆盖不完整，**missing** 表示当前没有可用证据。整体证据充分度可写为：

$$e_t = \text{clip} \left(\sum_m \lambda_m(g) \rho_m(o_t), 0, 1 \right) \quad (4-15)$$

式中， e_t ——时间步 t 的整体证据充分度；

该值进入 e_{suf} ，影响是否继续进入高代价步骤。若证据不足而候选下一步成本较高，运行时重排序会降低该候选；若已有充分低成本证据支持，则候选可以更合理地进入结构预测或目标评分。

将 G_{post} 替换静态效用中的目标匹配项后，得到包含后验证据的基础分 S_{post} 。运行时重排序定义为：

$$U_{\pi}(\pi, x_t) = \text{clip}(S_{\text{post}}(\pi) + \Delta(\pi, x_t), 0, 1) \quad (4-17)$$

式中， $S_{\text{post}}(\pi)$ ——替换后验目标项后的基础分；

其中 $\Delta(\pi, x_t)$ 是有界运行时修正项，取值范围控制在 $[-0.35, 0.35]$ 。其一般形式为：

$$\begin{aligned} \Delta(\pi, x_t) = & k_s(p_{\text{succ}} - 0.5)\text{Conf}(\pi) \\ & + k_e(2e_{\text{suf}} - 1) \max(\text{Conf}(\pi), F_s(\pi)) \\ & - k_f p_{\text{sf}}(1 - \text{RiskScore}(\pi)) + k_r r_{\text{rec}} \text{RecoveryScore}(\pi) \\ & - k_c b_{\text{press}}(1 - \text{CostScore}(\pi)) + k_a \text{ActionBias}(\pi, x_t) \end{aligned} \quad (4-19)$$

其中， $\text{Conf}(\pi)$ 表示候选置信度， $\text{RiskScore}(\pi)$ 和 $\text{CostScore}(\pi)$ 分别表示候选的风险质量分数和成本质量分数， $\text{RecoveryScore}(\pi)$ 表示候选的恢复友好度， $\text{ActionBias}(\pi, x_t)$ 表示候选与当前恢复动作偏好的匹配程度。直观来看，当结构性失败风险和预算压力升高时，算法会降低高成本、低可恢复性候选的排序；当成功概率、恢复余量和证据充分性较高时，算法会提高继续执行或低风险候选的排序。

运行时重排序有两个边界：一是只作用于已经通过可行性校验的候选，不能覆盖工具不存在、`schema` 错误、`I/O` 不闭合和安全阻断；二是修正幅度存在上界，避免一次异常观测完全覆盖静态目标匹配和工程可靠性判断。借助这两个边界，算法在适应运行时变化的同时仍能保持可复现和能够审计。

4.10 恢复动作与 HITL 映射

CEBRA-WP 将恢复动作限定为四类：`continue`、`patch_local`、`suffix_replan` 和 `stop`。四类动作与 FSM/HITL 的映射如表 4-3 所示。

表 4-3 恢复动作与 FSM / HITL 映射

动作	触发背景	系统效果	FSM/HITL 映射	审计结果
continue	成功概率较高、证据充分、风险和预算压力可接受	继续执行当前计划后续步骤	RUNNING 内自动推进或维持 PLANNED/RUNNING	记录动作效用与继续原因
patch_local	局部失败、重试耗尽、前缀仍有较高效、局部可修复性较高	生成局部 PlanPatch，替换参数或局部工具	WAITING_PATCH_CONFIRM 或策略允许的受控自动 patch	记录 patch 候选、失败上下文和应用结果
suffix_replan	结构性失败概率升高、后缀可靠性不足、前缀可保留	保留已验证前缀，替换未执行后缀	WAITING_REPLAN_CONFIRM	记录 replan 候选、前缀保留位置和新后缀
stop	成功概率低、预算压力高、恢复余量低、人工介入价值有限	生成终止型 ReplanCandidate	WAITING_REPLAN_CONFIRM NFIRM；接受后进入 FAILED	记录 terminal_reason 与已保留证据

动作效用由状态量和派生量共同计算。为便于表示，记 $s = p_{succ}$ ， $f = p_{sf}$ ， $r = r_{rec}$ ， $e = e_{suf}$ ， $b = b_{press}$ 。局部可修复性、证据可复用性、前缀可保留性、预算缓解度、目标重对齐收益和人工介入价值，分别记为 l_{patch} 、 e_{reuse} 、 p_{pres} 、 b_{relief} 、 g_{align} 和 v_{hitl} 。动作效用可写为：

$$U_{continue} = 0.38s + 0.14e + 0.12r - 0.22f - 0.14b \quad (4-21)$$

$$U_{patch_local} = 0.20s + 0.24r + 0.18l_{patch} + 0.12e_{reuse} - 0.14f - 0.12b \quad (4-22)$$

$$U_{suffix_replan} = 0.18(1 - s) + 0.20f + 0.16(1 - r) + 0.18p_{pres} + 0.14b_{relief} + 0.14g_{align} \quad (4-23)$$

$$U_{\text{continue}} = 0.38s + 0.14e + 0.12r - 0.22f - 0.14b \quad (4-21)$$

$$U_{\text{stop}} = 0.32(1 - s) + 0.24b + 0.18(1 - r) + 0.16s_{\text{safe}} + 0.10(1 - v_{\text{hit}}) \quad (4-24)$$

其中， s_{safe} 为安全终止性。这组效用函数区分了四类动作的不同偏好：**continue** 对较高成功概率、证据充分度和恢复余量更敏感；**patch_local** 更依赖局部可修复性、证据可复用性和较低结构性失败概率；**suffix_replan** 偏向于结构性失败概率较高、当前成功概率较低但前缀仍可保留的场景；**stop** 只有在继续价值低、预算压力高、恢复余量低且人工介入价值不足时才会成为强候选。

stop 不是用户主动取消，也不是执行异常崩溃，而是算法提出的终止型重规划候选。其载体为 `ReplanCandidate(replan_mode="terminal_stop")`，通过 `replan_confirm` 通道进入 `WAITING_REPLAN_CONFIRM`。只有在人工接受或策略显式允许时，系统才将任务推进到 `FAILED`；已完成前缀、产物和解释字段仍被保留为审计资产。

综合上述设计，CEBRA-WP 主流程可写为伪代码 4-1。

伪代码 4-1 CEBRA-WP 主流程

Input: g : 设计目标; C : 约束集合; K : ProteinToolKG; h_t : 执行历史;
 o_t : 当前观测; x_t : 轻量信念状态。

Output: $Decision_t$: 候选集合、默认候选、恢复动作、解释信息和证据引用。

```

1:  $Pi_{raw,t} \leftarrow \text{GenerateCandidates}(g, C, K, h_t)$ 
2:  $Pi_t \leftarrow \text{FeasibilityFilter}(Pi_{raw,t}, C, K, h_t)$ 
3: if  $Pi_t$  is empty and degraded_feasible candidates exist:
4:  $Pi_t \leftarrow$  degraded_feasible candidates
5: mark candidates as requiring HITL confirmation
6: for each  $pi$  in  $Pi_t$ :
7:  $S_{static}(pi) \leftarrow \text{StaticUtility}(pi, g, C, K)$ 
8:  $S_{post}(pi) \leftarrow \text{ReplaceObjectiveTerm}(S_{static}(pi), \text{PosteriorObjective}(pi, g, o_t))$ 
9:  $x_{t+1} \leftarrow \text{BeliefUpdate}(x_t, o_t, h_t)$ 
10: for each  $pi$  in  $Pi_t$ :
11:  $\Delta(pi, x_{t+1}) \leftarrow \text{RuntimeAdjustment}(pi, x_{t+1})$ 
12:  $U_{pi}(pi, x_{t+1}) \leftarrow \text{clip}(S_{post}(pi) + \Delta(pi, x_{t+1}), 0, 1)$ 
13:  $TopK_t \leftarrow \text{SelectDiverseTopK}(Pi_t, U_{pi}, \text{capability\_coverage})$ 
14:  $U_a \leftarrow \text{ComputeActionUtility}(\text{continue}, \text{patch\_local}, \text{suffix\_replan}, \text{stop})$ 
15:  $a_t \leftarrow \text{ApplyHardPrioritiesAndSelectAction}(U_a, x_{t+1}, h_t, C)$ 
16: return  $Decision_t(TopK_t, a_t, \text{explanations}, \text{evidence\_refs})$ 

```

算法第 1 至 8 行完成候选生成与评价，第 9 至 12 行依据运行时状态修正候选效用，第 13 行保留 Top-K 多样性，第 14 至 15 行选择恢复动作。第 15 行中的硬优先级包括安全阻断优先、schema/I/O/tool availability 违规淘汰、重试耗尽后优先局部修补、前缀可保留时优先后缀重规划等约束。借助这些优先级，动作选择服从系统控制边界，而不是简单按连续效用值排序。

4.11 策略组与实验可验证性

为了验证 CEBRA-WP 各组成机制的作用，系统将算法能力拆分为四组策略开关。四组策略不是四套独立系统，而是在同一在工程实现方面逐步打开静态评分、固定阈值门控、动态恢复和 Lite belief-state 的内部消融配置，如表 4-

4 所示。

表 4-4 四组消融策略与算法机制开关

策略组	静态评分	固定阈值门控	动态观测	Lite belief- state	Runtime rerank	恢复动 作效用	主要实验 问题
static_top1	启用	未启用	未启用	未启用	未启用	未启用	单一静态 最优候选 是否足以 完成任务
fixed _threshold _gate	启用	启用	有限使用	未启用	未启用	未启用	固定阈值 是否能控 制风险与 成本
dynamic _no_belief _stat	启用	启用	启用	未启用	部分启用	部分启 用	直接运行 时观测是 否已能支 撑恢复
lite_belief _state	启用	启用	启用	启用	启用	启用	显式状态 是否带来 更好的解 释、预算 感知和恢 复决策

该策略设计使第 7 章 可以分别考察：静态计划是否足够，固定阈值门控是否带来额外成本，以及在已有动态恢复时 Lite belief-state 的增量价值。本文不把 CEBRA-WP 论证为所有指标最优算法，而是验证其机制链路是否可执行、可观测、可审计。

4.12 数据契约与模块协作

系统以统一数据契约支撑算法与工程实现之间的衔接。核心契约关系如图

4-7 所示。

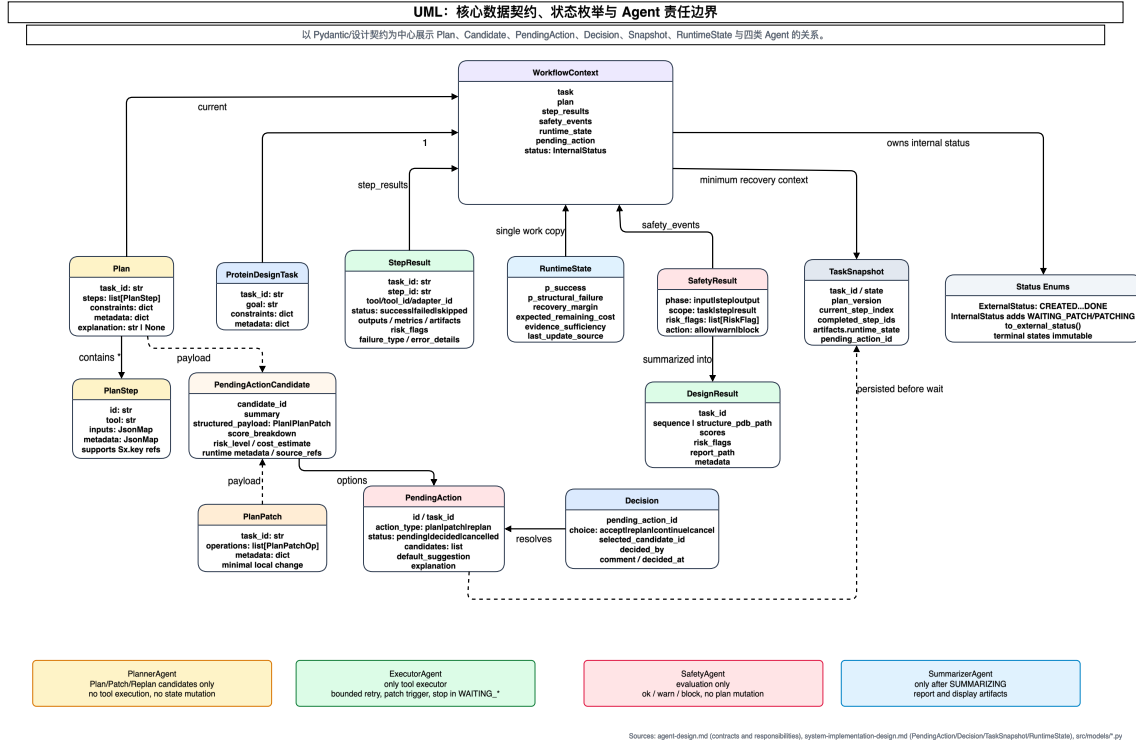


图 4-7 核心数据契约 UML

ProteinDesignTask 记录 task_id、goal 和约束字段；Plan 由 PlanStep 组成，并用 S{id}.{field} 建立依赖；StepResult 保存执行状态、输出、指标和失败信息；PendingAction/Decision 管理人工确认；TaskSnapshot 保存可恢复状态。

具体实例走查：t1_trpcage_denovo_short_peptide 数据流

基于 thesis-final-v1-001 的 Trp-cage-like 短肽任务配置与运行结果，展示任务对象、计划执行和结果汇总之间的字段传递。

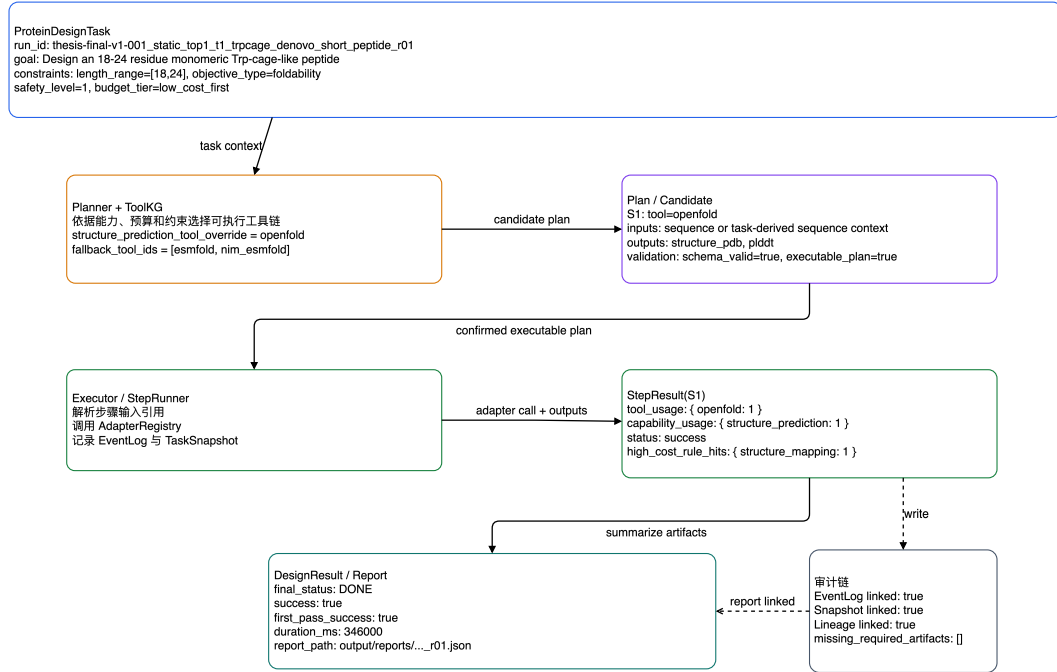


图 4-8 t1 任务实例走查

图 4-8 展示了典型任务 t1 的贯穿示例。

该示例从任务创建开始，依次经过 Planner 生成候选计划、用户确认、Executor 调用工具、StepResult 写入、RuntimeState 更新和必要的恢复候选触发，最后由 Summarizer 汇总为 DesignResult。CEBRA-WP 在这里不是孤立算法模块，而是嵌入任务生命周期的规划与恢复层：算法输出通过 PendingAction、PlanPatch、ReplanCandidate 和 RuntimeState 等契约进入系统，任务状态仍由 FSM 和 Workflow 控制。

4.13 本章小结

本章给出系统总体设计和 CEBRA-WP 算法定义。系统采用五层架构，并以 FSM、HITL、ProteinToolKG 和六阶段 workflow 约束任务推进；算法层则定义候选生成、硬可行性过滤、静态评分、Lite belief-state 更新、运行时重排序和恢复动作选择。

第5章 系统实现

本章在第四章系统设计的基础上说明工程实现方式。本文不提出新的底层蛋白质生成模型，而是把已有工具组织为可执行、可恢复、可审计的智能 workflow。因此，本章主要讨论任务接入、候选计划生成、HITL 确认、工具执行、运行时恢复和结果审计等机制如何落实到代码模块和接口中。

蛋白质设计相关能力主要通过 AlphaFold/OpenFold、ESMFold、ProteinMPNN、ProtGPT2、Biopython 等工具或工具适配器接入，这些工具本身已有成熟研究基础^[2,4-6,10,18]。因此，实现工作的重点在于把这些异构能力组织为可追踪、可恢复、可解释的工作流。

从工作量看，本文的实现难点不在于单独调用某一个蛋白质设计工具，而在于把自然语言任务、工具能力、输入输出字段、运行状态、人工决策和审计证据放到同一条可恢复链路中。实际开发中需要同时维护后端 API、前端工作台、工具适配器、FSM 状态迁移、事件日志、快照恢复和实验脚本；任何一个环节的状态字段或 I/O 契约不一致，都会影响后续候选生成、执行和实验统计。本文通过统一数据模型、显式状态机、适配器抽象和证据日志来降低这种耦合。

5.1 技术选型与工程结构

后端选用 Python 3.12、FastAPI 和 Pydantic。Python 便于封装生物信息脚本和模型服务，FastAPI 提供路由与响应模型，Pydantic 用来定义任务、计划、步骤结果、决策和报告等相关数据契约。

前端选用 React、TypeScript 和 Vite 搭建轻量 Web 工作台。它不再单独保存业务状态，而是通过后端 API 读取任务记录、待决策对象、事件时间线和报告结果。这种处理方式可以减少前端界面与后端 workflow 状态分叉，使 Web 页面主要承担起运行时状态展示和交互入口的作用。

workflow 控制采用自定义 Workflow/FSM，而非外部流程引擎。Nextflow 等工具适合可复现计算^[11]，但本文需显式处理 WAITING_*、PendingAction、

Decision、RuntimeState 和 TaskSnapshot，因此将全局状态迁移和人工确认保留在自定义运行时中。

表 5-1 给出了后端核心模块与论文架构层之间的对应关系。该表说明系统实现并非简单的文件目录堆叠，而是围绕任务接入、数据契约、 workflow 控制、工具适配和审计恢复形成分层实现。

表 5-1 后端核心模块与论文架构层对应关系

实现目录	主要职责	对应设计层/模块	说明
src/api/	FastAPI 入口、任务接口、HITL 接口、前端静态入口	输入交互层、任务接入模块	API 是任务、事件、报告和人工决策的统一边界。
src/models/	Pydantic 契约、状态枚举、任务记录	核心数据契约	任务、计划、步骤结果、PendingAction、RuntimeState 等对象均受结构化约束。
src/agents/	Planner、Executor、Safety、Summarizer	多 Agent 协作模块	各 Agent 只承担明确职责，不直接越权改变全局状态。
src/workflow/ w/	WorkflowContext、PlanRunner、StepRunner、RuntimeEvaluator、恢复与快照	workflow 控制层、CEBRA-WP 工程承载	FSM、HITL、重试、恢复、runtime rerank 和快照的主要实现位置。
src/adapters/ /	ToolAdapter、AdapterRegistry、具体工具适配器	工具执行与资源接入层	屏蔽本地工具、远程 REST 服务和脚本差异。

表 5-1（续表）

实现目录	主要职责	对应设计层/模块	说明
src/kg/	ProteinToolKG JSON 与查询逻辑	ProteinToolKG	为候选生成提供工具 能力、I/O、成本和安 全约束。
src/storage/	事件日志、快照、文 件产物管理	审计与恢复支撑	EventLog 和 TaskSnapshot 支撑恢 复、复核和实验证据 提取。

5.2 任务接入与后端 API 实现

5.2.1 渐进式任务录入

Task Intake 将自然语言目标拆分为草稿创建、字段补充、场景预检查和确认创建，避免自由文本直接进入高代价执行。该链路补齐长度、任务类型、模板、预算、安全限制和评分方式等关键字段。

后端提供 /task-intakes/schema、/task-intakes、/task-intakes/{id} 和 /task-intakes/{id}/confirm 等接口。字段注册表由后端返回，前端 Task Builder 再依据字段类型、必填状态和分组信息渲染表单。用户补充字段后，系统执行场景门控和安全预检查；只有经过确认的结构化任务规格，才会进入正式任务创建路径。

正式任务创建接口支持 goal、query 和 confirmed_task_spec 三种入口，但三者必须互斥。代码 5-1 任务创建请求的互斥入口校验展示了任务创建请求模型中的互斥校验。该校验用于保证任务来源清晰，避免自由文本、兼容入口和已确认结构化规格混杂进入同一个任务。

代码 5-1 任务创建请求的互斥入口校验

```

1: class TaskCreateRequest(BaseModel):
2:     goal: Optional[str] = Field(
3:         None,
4:         description="蛋白质设计任务目标(自然语言)",
5:     )
6:     query: Optional[str] = Field(
7:         None,
8:         description="兼容自由文本入口；会收敛为 intake",
9:     )
10:    confirmed_task_spec: Optional[ConfirmedTaskSpec] = Field(
11:        None,
12:        description="已经确认的结构化任务输入",
13:    )
14:    constraints: Dict[str, Any] = Field(
15:        default_factory=dict,
16:        description="结构化约束",
17:    )
18:    metadata: Dict[str, Any] = Field(default_factory=dict)
19:
20:    @model_validator(mode="after")
21:    def _validate_creation_mode(self) -> "TaskCreateRequest":
22:        modes = [
23:            self.goal is not None,
24:            self.query is not None,
25:            self.confirmed_task_spec is not None,
26:        ]
27:        if not any(modes):
28:            raise ValueError(
29:                "one of goal, query, or confirmed_task_spec is required"
30:            )
31:        if sum(modes) > 1:

```

```

32:         raise ValueError(
33:             "choose exactly one of goal, query, or confirmed_task_spec"
34:         )
35:     return self

```

代码 5-1 任务创建请求的互斥入口校验 对应第四章中“任务输入需要结构化确认”的设计要求。该校验不直接提高蛋白质设计模型的能力，但可以避免任务入口语义不清导致后续计划生成和审计困难。

5.2.2 任务生命周期接口

任务生命周期接口暴露任务状态、事件和报告。GET /tasks/{task_id} 返回外部状态、内部状态、目标、约束、计划、结果和安全事件，便于前端展示与后端恢复逻辑分离。

GET /tasks/{task_id}/events 直接读取事件日志，而非只依赖内存任务表。只要日志存在，服务重启后仍可还原状态迁移、步骤执行、等待进入和决策应用等过程。

5.2.3 HITL 接口

HITL 机制通过 PendingAction 和 Decision 建模。GET /pending-actions 返回待审查任务摘要，GET /pending-actions/{id} 返回候选详情，POST /pending-actions/{id}/decision 接收用户决策。候选详情中包含候选数量、默认建议、评分分解、运行时状态摘要、风险等级、成本估计和解释字段。前端因此可以展示“为什么推荐该候选”，而不是只提供一个缺少解释的确认按钮。

当用户提交 Decision 后，后端会验证 PendingAction 是否属于当前任务、是否仍处于待决策状态、所选候选是否存在。验证通过后，后端根据 action type 将决策应用到 plan、局部修补或重规划，并写入决策事件和等待退出事件。这一过程保证人工决策属于 workflow 状态，而不是前端临时 UI 状态。

5.3 前端工作台实现

前端工作台包含 Dashboard、Task Builder、Task Detail 和 Event Timeline 四类主要页面。Dashboard 展示任务列表以及、状态摘要和工具能力 readiness；Task Builder 负责进行任务录入、字段补充和确认；Task Detail 聚合任务状态、候选审查、报告与结构产物入口；Event Timeline 用于查看任务生命周期事件。

前端启动时从后端注入的 bootstrap payload 获取当前视图和任务 ID，再通过 API 拉取任务、待决策对象、事件和报告。这种实现方式使前端不需要自行推导 FSM 状态，也不会在页面刷新或任务切换时产生独立状态源。

在 Task Detail 页面中，PendingReviewWorkspace 是 HITL 的主要交互区域。当任务进入 WAITING_* 状态时，前端展示候选方案对比、评分分解、运行时上下文和决策表单。用户提交决策后，前端仅向后端发送结构化 Decision，具体的计划应用、状态迁移、快照写入和事件记录均由后端 workflow 模块完成。

当前前端的结构展示区域主要提供结构文件路径、报告浏览和可视化入口。并不具备提供完整的三维结构分析。

5.4 运行时与执行引擎

图 5-1 展示了运行时执行序列：API 创建任务，Workflow 建立上下文，Planner 生成候选，Executor 在确认后执行 PlanStep，StepRunner 通过 ToolAdapter 解析输入、调用工具、校验输出并写回结果。

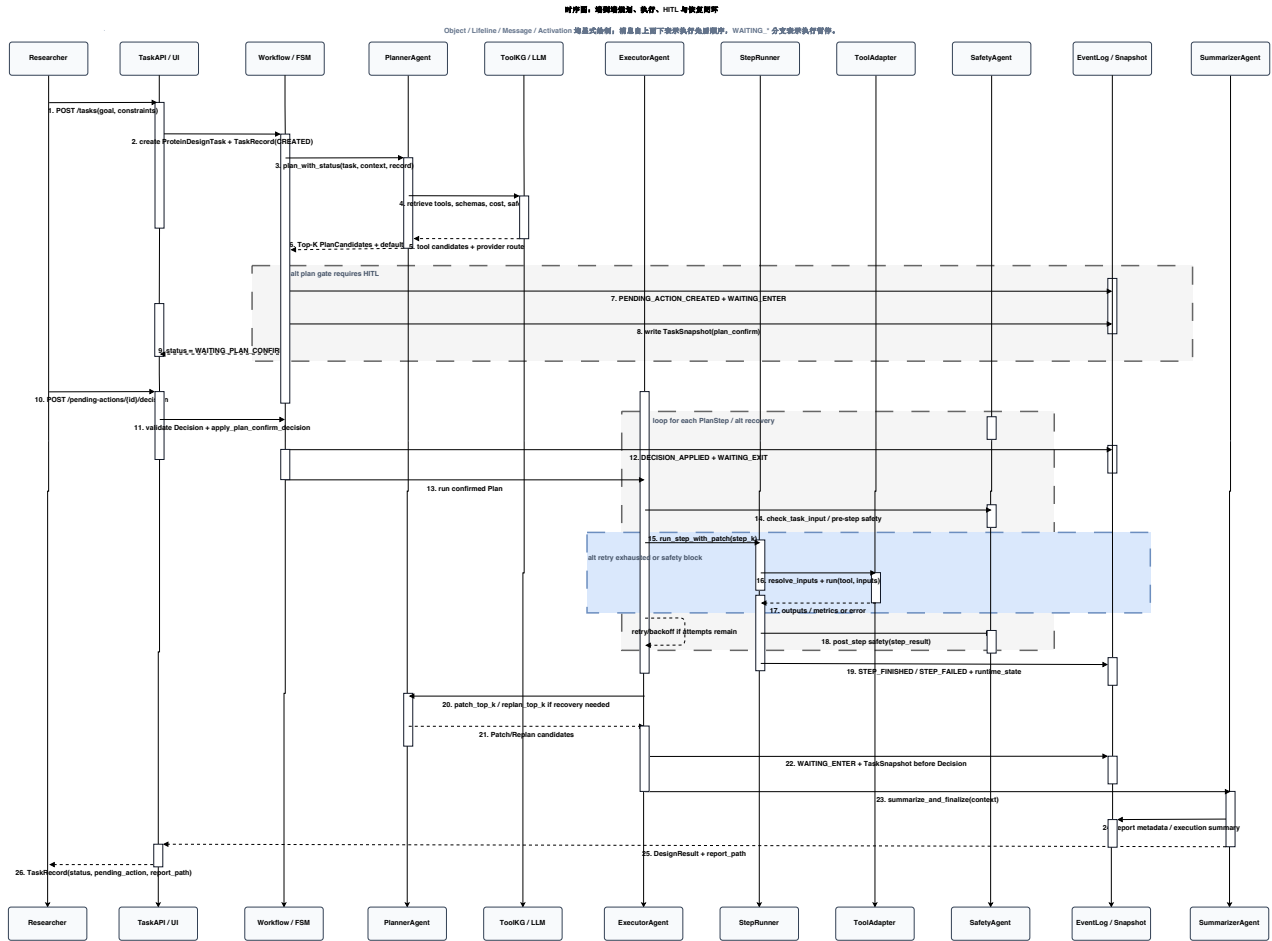


图 5-1 运行时执行序列

图 5-1 重点说明了 Planner 与 Executor 的边界: Planner 生成候选并查询工具知识，不直接执行工具；Executor 只执行已确认的 PlanStep，并通过 StepRunner 和 ToolAdapter 与外部工具交互。这样，候选生成、人工确认、工具执行和审计记录可以被分别追踪。

5.4.1 WorkflowContext

WorkflowContext 是单个任务运行期间的上下文对象，保存任务、当前计划、步骤结果、安全事件、RuntimeState、最终结果和 PendingAction。运行时状态更新不由各模块分散修改，而是通过上下文提供的统一入口触发。代码 5-

2 WorkflowContext 中的 RuntimeState 更新入口 展示了 StepResult 和 SafetyResult 写入时如何触发 RuntimeState 更新。

代码 5-2 WorkflowContext 中的 RuntimeState 更新入口

```

1: def add_step_result(self, result: StepResult) -> None:
2:     self.step_results[result.step_id] = result
3:     if not runtime_policy_uses_belief_state(self.task):
4:         return
5:     self.apply_runtime_state_update(
6:         step_result=result,
7:         failure_context=extract_failure_context(result),
8:     )
9:
10: def add_safety_event(self, event: SafetyResult) -> None:
11:     self.safety_events.append(event)
12:     if not runtime_policy_uses_belief_state(self.task):
13:         return
14:     self.apply_runtime_state_update(safety_result=event)

```

该实现对应第四章中 Lite belief-state / 轻量信念状态的设计。系统只有在任务启用相关 runtime policy 时才更新 RuntimeState，从而支持 static、dynamic 和 lite belief-state 等策略在同一代码基中切换。

5.4.2 计划与步骤执行器

PlanRunner 负责计划级执行和 FSM 推进。它在执行前检查任务与计划一致性，按顺序执行 PlanStep；可修复失败进入 WAITING_PATCH_CONFIRM，结构性失败或安全阻断进入 WAITING_REPLAN_CONFIRM。

StepRunner 是单步执行的最小单元。它负责有界重试、安全检查、工具适配、输入解析、工具调用、输出校验和错误归一化。代码 5-3 StepRunner 的有界重试微循环 展示了 StepRunner 的有界重试微循环。该循环只对可重试失败进行重试；当失败不可重试或重试耗尽时，StepRunner 返回结构化 StepResult，

由上层恢复逻辑决定是否进入 `patch_local`、`suffix_replan` 或 `stop`。

代码 5-3 StepRunner 的有界重试微循环

```

1: def run_step(self, step: PlanStep, context: WorkflowContext) -> StepResult:
2:     """执行单个 PlanStep（带静态重试），返回最终 StepResult"""
3:     last_result: StepResult | None = None
4:     attempt_logs: list[AttemptRecord] = []
5:     retried_any = False
6:
7:     for attempt_idx in range(1, self._retry_policy.max_attempts + 1):
8:         result = self._run_once(step, context)
9:         self._annotate_attempt_meta(result, attempt_idx)
10:        attempt_logs.append(self._build_attempt_record(result, attempt_idx))
11:        last_result = result
12:
13:        if result.status == "success":
14:            return self._finalize_success(result, attempt_logs)
15:
16:        failure_type = self._normalize_failure_type(result.failure_type)
17:        retried_any = retried_any or attempt_idx > 1
18:        exhausted = attempt_idx >= self._retry_policy.max_attempts
19:        if failure_type is None or not is_retryable_failure(failure_type):
20:            return self._finalize_failure(result, attempt_logs, retried_any,
exhausted)
21:        if exhausted:
22:            return self._finalize_failure(result, attempt_logs, retried_any,
True)
23:
24:        backoff_ms = self._retry_policy.backoff_ms_for(attempt_idx)
25:        if backoff_ms > 0:
26:            self._sleep_fn(backoff_ms / 1000)
27:
28:        return self._finalize_failure(last_result, attempt_logs, retried_any, True)

```

该实现使工具异常不会直接向上层模块扩散。无论底层问题来自本地脚本失败、远程服务超时，还是输出字段缺失，PlanRunner 和 CEBRA-WP 接收到的都是统一的 `failure_type`、`attempt history` 和错误详情。

这一部分也是实现过程中最容易出问题的地方。蛋白质设计工具的输入输出并不天然统一，有的返回结构文件路径，有的返回序列或评分，有的失败只表现为超时或字段缺失。若让上层直接理解每种错误，恢复逻辑会很快失控。因此本文将失败归一化为 `failure_type`、`attempt history`、`risk flags` 和 `metrics`，再由 PlanRunner 与 CEBRA-WP 判断重试、局部修补、后缀重规划或终止。这种取舍牺牲了一部分工具细节展示，但换来了更稳定的状态控制和可审计的恢复路径。

5.4.3 等待状态、审计与快照

恢复闭环的关键在于在进入等待状态前先保存待决策对象和审计信息。代码 5-4 进入 WAITING 状态前写入 PendingAction 与审计信息 展示了 `enter_waiting_state` 的核心逻辑：状态迁移前，系统会先校验等待状态与 PendingAction 是否匹配，再写入 `context`、`TaskRecord` 和事件日志。这样即使系统在等待人工确认期间中断，恢复后也能还原当前待决策对象、候选集合和默认建议。

代码 5-4 进入 WAITING 状态前写入 PendingAction 与审计信息

```

1: def enter_waiting_state(
2:     context: WorkflowContext,
3:     record: TaskRecord | None,
4:     pending_action: PendingAction,
5:     to_status: InternalStatus,
6:     *,
7:     reason: Optional[str] = None,
8:     event_logger: EventLogger | None = None,
9:     snapshot_writer: SnapshotWriter | None = None,
10: ) -> None:
11:     _validate_waiting_transition(context, pending_action, to_status, record)
12:
13:     prev_status = to_external_status(context.status)
14:     new_status = _to_external_waiting_status(to_status)
15:
16:     context.pending_action = pending_action
17:     if record is not None:
18:         record.pending_action = pending_action
19:
20:     log_handler = event_logger or _default_event_logger
21:     log_handler(
22:         {
23:             "event": "PENDING_ACTION_CREATED",
24:             "task_id": context.task.task_id,
25:             "pending_action_id": pending_action.pending_action_id,
26:             "action_type": pending_action.action_type.value,
27:             "candidate_ids": [c.candidate_id for c in
pending_action.candidates],
28:             "default_suggestion": pending_action.default_suggestion,
29:             "default_recommendation":
pending_action.default_recommendation,

```

```
30:         }
```

```
31:     )
```

该代码说明，HITL 在系统中被实现为受控运行时状态，而不是前端组件上的临时按钮。PendingAction、Decision、EventLog 和 TaskSnapshot 一起构成了可恢复、可复核的 HITL 机制。

5.5 CEBRA-WP 的工程落点

CEBRA-WP 在工程实现中不是一个单独的 Agent，而是分布在候选生成、RuntimeState 更新、候选重排、恢复动作映射和候选解释几个环节。Planner 生成候选及其静态评分，WorkflowContext 维护 RuntimeState，RuntimeEvaluator 计算 runtime adjustment 和 action utility，PlanRunner 则将动作建议映射为 continue、patch_local、suffix_replan 或 terminal_stop 等工作流行为。

代码 5-5 RuntimeEvaluator 候选重排 展示了 RuntimeEvaluator 对候选进行运行时评估和重排的入口。策略模式禁用重排时，系统返回静态默认候选；缺少 RuntimeState 时，仅执行 passthrough；当 Lite belief-state / 轻量信念状态可用时，系统根据 RuntimeState 计算 runtime adjustment，并按 final score 重新排序。

代码 5-5 RuntimeEvaluator 候选重排

```

1: def evaluate_candidates(
2:     self,
3:     candidates: list[PendingActionCandidate],
4:     runtime_state: RuntimeStateSchema | Mapping[str, object] | None,
5: ) -> RuntimeEvaluation:
6:     """对每个候选计算 runtime_adjustment 与 final_score, 按 final_score 降
序重排。"""
7:     if not candidates:
8:         return RuntimeEvaluation(policy_mode=self._policy_mode)
9:
10:    state = _coerce_state(runtime_state)
11:    static_default = _top_static_candidate(candidates)
12:
13:    if policy_disables_rerank(self._policy_mode):
14:        return RuntimeEvaluation(
15:            candidates=list(candidates),
16:            static_default_id=static_default,
17:            reranked_default_id=static_default,
18:            rerank_applied=False,
19:            policy_mode=self._policy_mode,
20:        )
21:
22:    if state is None:
23:        reranked = [_apply_passthrough(c) for c in candidates]
24:        reranked.sort(key=_final_score_key, reverse=True)
25:        top = reranked[0].candidate_id if reranked else None
26:        return RuntimeEvaluation(
27:            candidates=reranked,
28:            static_default_id=static_default,
29:            reranked_default_id=top,
30:            rerank_applied=False,

```

```

31:         policy_mode=self._policy_mode,
32:     )

```

该实现对应第四章中 CEBRA-WP 的运行时的重排序环节。需要强调的是，RuntimeEvaluator 只会在已满足硬可行性约束的候选之间调整排序，不会绕过工具存在性、schema、I/O、安全和预算硬约束。

图 5-2 从泳道视角说明各模块协作关系。科研人员通过 Web UI 创建任务并提交决策，Task API 负责请求与响应边界，Planner 生成候选，Executor 调度 StepRunner 和 ToolAdapter，Safety 提供风险信号，Storage 固化事件和快照。CEBRA-WP 贯穿在这一过程中：候选生成后参与候选重排，失败或风险出现时参与恢复动作选择，并通过 PendingAction 把解释信息呈现给人工决策者。

蛋白质设计工具链具有明显异构性：有的工具是本地 Python 脚本，有的工具依赖命令行环境，有的工具通过远程 REST 服务提供能力。为了避免 Executor 绑定具体工具实现，系统通过 BaseToolAdapter 定义统一工具接口。代码 5-6 ToolAdapter 抽象接口 展示了适配器抽象基类的核心方法。

代码 5-6 ToolAdapter 抽象接口

```

1: class BaseToolAdapter(ABC):
2:     """ToolAdapter 抽象基类，统一工具调用入口"""
3:
4:     tool_id: str
5:     adapter_id: str | None = None
6:
7:     @abstractmethod
8:     def resolve_inputs(
9:         self, step: PlanStep, context: WorkflowContext
10:    ) -> Dict[str, Any]:
11:         """将 PlanStep.inputs 解析为工具实际输入"""
12:
13:     @abstractmethod
14:     def run_local(
15:         self, inputs: Dict[str, Any]
16:    ) -> Tuple[Dict[str, Any], Dict[str, Any]]:
17:         """本地执行工具并返回 (outputs, metrics)"""
18:
19:     def run_remote(
20:         self,
21:         inputs: Dict[str, Any],
22:         output_dir: Optional[Path] = None,
23:    ) -> Tuple[Dict[str, Any], Dict[str, Any]]:
24:         raise NotImplementedError(
25:             f"{self.__class__.__name__} does not support remote execution"
26:         )

```

该接口将输入解析、执行模式和输出指标统一到同一抽象层。对于 `Executor` 而言，不同工具都表现为“接收结构化输入，返回 `outputs` 和 `metrics`”的适配器。具体工具内部是调用 `Biopython`、远程 `OpenFold` 服务还是序列生成模型，对上层 workflow 保持透明。

`AdapterRegistry` 维护 `tool_id` 到适配器实例的映射。`StepRunner` 执行某个 `PlanStep` 时，先通过 `tool_id` 找到适配器，再调用 `resolve_inputs` 解析常量输入和上游步骤引用。若工具未注册、上游引用缺失或输出类型不匹配，系统返回结构化失败，不会静默跳过错误或继续执行。

`ProteinToolKG` 以 JSON 形式保存工具能力、输入输出、兼容关系、成本、安全级别和版本信息。`Planner` 在候选生成时查询 `ProteinToolKG`，根据任务所需能力组合工具链，过滤不可用工具、预算超限工具或 I/O 闭包不成立的候选。这一实现使新增工具主要依靠更新 `ProteinToolKG` 和注册适配器完成，而不需要修改 `Planner` 的核心流程。

5.7 本章小结

本章从工程角度说明了系统实现方式。系统后端以 `Python`、`FastAPI` 和 `Pydantic` 为基础，前端以 `React`、`TypeScript` 和 `Vite` 构建工作台；workflow 控制采用自定义 `Workflow/FSM`，使 `HITL`、快照恢复和事件审计保持显式可控。实现过程中的主要工作量集中在跨模块契约统一、工具适配、等待态恢复和证据链整理，而不是单一页面或单一模型接口。

本章还说明了系统实现中的主要难点和取舍：自然语言任务需要先转为结构化规格，候选计划必须满足工具 I/O 闭包，运行时失败需要被归一化为可恢复的状态，人工确认必须进入 `FSM` 和审计日志。本文通过 `Task Intake`、`WorkflowContext`、`PlanRunner`、`StepRunner`、`PendingAction`、`Snapshot` 和 `ToolAdapter` 等模块，将这些问题拆分到相对清晰的边界内。

第六章将围绕 API 合约、`FSM/HITL`、快照恢复、前端与 `CLI` 可用性，以及 `retry`、局部修补、后缀重规划和安全边界展开验证。

第6章 系统测试与验证

本章在系统实现基础上验证原型系统的功能正确性和工程可用性，重点检查任务接入、状态控制、HITL、快照恢复、工具适配、安全边界和审计追踪等关键链路。与第七章的策略消融不同，本章只回答系统能否稳定承载后续实验，不把验证结果扩大为生物学功能验证。

6.1 测试策略与验证目标

测试采用单元测试、集成测试、接口验证、前端 smoke 测试和端到端运行相结合的方式。单元测试关注数据契约、状态迁移和恢复动作；集成测试关注 Planner、Executor、Safety、RuntimeEvaluator 与 Storage 的协作；API、Web 和 CLI 验证实际使用入口。

测试用例 TC-S01 至 TC-S13 覆盖环境与能力就绪、任务录入、候选生成、HITL、FSM、快照、前端与 CLI、失败恢复、安全边界和端到端执行。表 6-1 汇总了覆盖范围、执行结果和证据。

表 6-1 系统测试用例覆盖矩阵

用例	测试类别	覆盖验证点	执行结果	核心证据
TC-S01	环境与能力就绪	SV-01、SV-27	通过	EVD-API-01、EVD-API-02

表 6-1（续表）

用例	测试类别	覆盖验证点	执行结果	核心证据
TC-S02	API 合约与任务 录入	SV- 02/03/04/07/25/26	通过	EVD-TEST-01、 EVD-API-03 至 EVD-API-08
TC-S03	计划候选生成	SV-08/09/10	通过	EVD-TEST-02、 EVD-LOG-04
TC-S04	HITL 决策 一致性	SV-10/11/12/13/15	通过	EVD-TEST-02、 FIG-SV-15
TC-S05	FSM 状态 迁移	SV-14/23/24	通过	EVD-TEST-02、 EVD-LOG-03
TC-S06	快照持久化 与恢复	SV-14/28/29	通过	EVD-TEST-02、 EVD-LOG-01 至 EVD-LOG-03
TC-S07	Web 前端可 用性	SV-01、SV-30	通过	FIG-SV-01 至 FIG-SV-18、EVD-TEST-01
TC-S08	CLI 可用性	SV-02/03/25/26/30	部分通过	EVD-CLI-01 至 EVD-CLI-04、EVD-TEST-04
TC-S09	端到端任务 流程	SV-16/25/26	通过	EVD-EXP-01、 EVD-LOG-05
TC-S10	异常输入与 安全边界	SV- 04/05/06/11/13/21/24	通过	EVD-TEST-03
TC-S11	工具链执行 与 I/O	SV-16/17	通过	EVD-TEST-03、 EVD-EXP-01

表 6-1（续表）

用例	测试类别	覆盖验证点	执行结果	核心证据
TC-S12	失败恢复流程	SV-18/19/20/22	通过	EVD-TEST-03 、 EVD-LOG-01 至 EVD-LOG-03
TC-S13	恢复止损与审计 链路	SV-21/23/25	通过	EVD-TEST- 02/03、EVD-LOG- 03/04

由表 6-1 可见，13 个测试用例中 12 个通过，1 个部分通过。部分通过项为 CLI 可用性验证：intake schema 和 task show 等核心命令可用，timeline show 与 report show 当前仍输出 usage 提示，因此在终稿中作为实现限制说明。

附录 A 保留每条测试用例的设计目的、触发方式、预期结果和证据边界；正文后续小节只说明与系统正确性论证直接相关的关键路径。

本章使用的证据材料按编号前缀组织，如表 6-2 所示。正文仅保留必要证据编号，完整文件路径由系统验证材料索引维护。FIG-SV-01 至 FIG-SV-18 属于前端验证截图证据，本章不将其作为正式“图 6-x”编号插图，以避免正文图号与证据截图编号混用。

表 6-2 系统验证证据类型索引

证据前缀	证据类型	当前数量/范围	主要支撑内容
EVD-API	API 响应 JSON	8 项	健康检查、能力 readiness、任务详情、事件、报告和 pending action 接口
EVD-TEST	Pytest 执行日志	4 组	API、Web smoke、FSM/HITL/快照、安全恢复和 CLI 测试结果
EVD-CLI	CLI 输出日志	4 项	intake schema 与 task show 可用， timeline/report 子命令为当前限制
FIG-SV	前端截图	18 张	Dashboard、Task Builder、Task Detail、 Timeline 页面可用性
EVD-LOG	EventLog/ Snapshot/Report 样本	8 组	状态迁移、恢复闭环、terminal_stop、等待态快照与报告产物
EVD-EXP	实验矩阵与端到端 运行聚合	4 项	smoke/clean run 记录、工具链执行结果 和端到端流程证据

6.2 API 服务与任务录入验证

API 服务验证表明，/health 能返回服务状态、任务数量、能力数量和数据目录，/capabilities/readiness 能区分 ready、degraded 和 unavailable 能力，并给出错误类别与恢复建议。任务录入验证确认字段注册表、自由文本草稿、confirm 必填字段、goal、query 与 confirmed_task_spec 互斥等入口约束均能生效。

6.3 计划候选与 HITL 验证

计划候选生成验证确认，PlannerAgent 输出的 PlanCandidate 包含 candidate_id、score_breakdown、risk_level、cost_estimate、explanation 和 source_refs 等必需字段。Planner 只负责候选生成与解释，不直接执行工具，也不越权改变任务状态。

HITL 验证覆盖 PendingAction 创建、Decision 提交和异常边界。系统进入 WAITING_PLAN_CONFIRM、WAITING_PATCH_CONFIRM 或 WAITING_REPLAN_CONFIRM 前必须写入对应 PendingAction；接受决策时必须提供合法候选，重复提交或跨任务提交会被拒绝。

事件日志显示，PENDING_ACTION_CREATED、WAITING_ENTER、DECISION_SUBMITTED、DECISION_APPLIED 和 WAITING_EXIT 构成完整人工决策审计链。等待态下 Executor 不继续调用工具，决策生效后系统再按 FSM 规则迁移。

6.4 状态机、快照与恢复验证

FSM 验证覆盖 CREATED、PLANNING、PLANNED、RUNNING、WAITING_*、SUMMARIZING、DONE、FAILED 和 CANCELLED 等关键状态。测试确认合法迁移可以执行，非法迁移会被拒绝；DONE、FAILED 和 CANCELLED 作为终态，进入后不能被普通状态更新覆盖。

快照验证对应 TC-S06。系统要求进入任意 WAITING_* 状态前完成 PendingAction 写入、事件日志记录和 TaskSnapshot 保存；恢复到等待态后，系统不会自动继续执行，而是继续等待人工 Decision。

Lite belief-state / 轻量信念状态相关状态量写入 snapshot artifacts 的 RuntimeState，Plan 继续保存步骤、输入输出和约束定义。该隔离避免运行时估计污染计划语义，也便于比较不同策略配置。

6.5 前端与 CLI 可用性验证

系统的工程可用性不仅取决于后端接口，也取决于用户实际接触的交互入口。TC-S07 验证 Web 工作台，TC-S08 验证 CLI。表 6-3 汇总了两类入口的覆盖范围、证据和限制。

表 6-3 前端与 CLI 可用性证据汇总

入口	覆盖范围	证据编号	结论	限制
Web Dashboard	任务列表、状态摘要、能力提示	FIG-SV-01、FIG-SV-02、EVD-TEST-01	通过	以截图和 smoke test 为主，复杂浏览器兼容性仍需另行验证
Task Builder	字段注册表、草稿补充、安全预检查、任务确认	FIG-SV-03 至 FIG-SV-11、EVD-API-03	通过	字段语义由 API schema 支撑，截图只作为交互证据。
Task Detail	任务状态、运行上下文、候选决策、报告与结构区域	FIG-SV-12 至 FIG-SV-17、EVD-API-04、EVD-API-06	通过	结构区域应表述为产物入口与展示区域，不扩大为完整三维结构分析平台。

表 6-3（续表）

入口	覆盖范围	证据编号	结论	限制
Event Timeline	状态迁移、步骤完成、等待进入、决策应用、等待退出	FIG-SV-18、 EVD-API-05	通过	当前以单任务样本展示全生命周期。
CLI	intake schema、task show、 timeline/report 命令	EVD-CLI-01 至 EVD-CLI-04、 EVD-TEST-04	部分通过	timeline 和 report 子命令当前仅输出 usage，需作为限制说明。

Web 工作台验证覆盖 Dashboard、Task Builder、Task Detail 和 Event Timeline。前端 smoke 测试与截图证据表明，用户可以查看任务状态、补充任务字段、审查候选、读取报告并追踪事件。

CLI 验证显示，intake schema --json 能输出字段注册表，task show 能展示任务 ID、状态、15 条能力 readiness 和恢复建议。CLI 自动化测试覆盖 16 个用例并全部通过；但 timeline show 和 report show 尚未提供完整展示能力，因此 TC-S08 标记为部分通过。

6.6 失败恢复与安全验证

蛋白质设计 workflows 中，工具失败、输出缺失、远程服务不可达和安全风险都可能导致任务中断。本文系统将失败处理拆分为有界重试、局部修补、后缀重规划和终止止损几个层次。TC-S12 和 TC-S13 分别验证恢复流程和止损审计，TC-S10 验证安全边界。表 6-4 汇总了恢复与安全相关的主要证据。

表 6-4 恢复与安全边界验证证据表

验证主题	覆盖用例	关键行为	主要证据	结论边界
有界重试	TC-S12	可重试失败进入有限重试， 耗尽后交给恢复逻辑	EVD-TEST-03、 EVD-LOG-01	证明 <code>retry</code> 机制可达， 不等同于所 有工具失败 都可恢复。
局部修补	TC-S12	<code>retry exhausted</code> 后生成 <code>patch_local</code> 候选并进入 <code>WAITING_PATCH_CONFIRM</code>	EVD-TEST-03、 EVD-LOG-01、 EVD-LOG-02	证明局部修 补路径可执 行，具体成 功依赖失败 类型。
后缀重规划/止 损	TC-S12、TC- S13	结构性失败或 <code>terminal_stop</code> 通过 <code>FSM</code> 进入重规划确认 或 <code>FAILED</code>	EVD-TEST-02、 EVD-TEST-03、 EVD-LOG-03	证明 <code>stop</code> 不绕过 <code>FSM/HITL</code> 。
安全 <code>block</code>	TC-S10、TC- S13	<code>forbidden motif</code> 在 <code>pre-step</code> 阶段阻断工具调用	EVD-TEST-03	矩阵实验未 充分触发 <code>safety</code> <code>block</code> ，结 论主要来自 <code>focused</code> <code>tests</code> 。

表 6-4（续表）

验证主题	覆盖用例	关键行为	主要证据	结论边界
安全 warn	TC-S10	warn 放行但记录风险标记和 安全事件	EVD-TEST-03、 FIG-SV-10	证明风险可 记录，不宣 称自动生物 安全判定完 备。

恢复验证确认，StepRunner 会对可重试失败执行有界重试；重试耗尽后返回结构化失败，由上层决定 patch_local 或 suffix_replan。确定性样本能够触发 patch_local 并进入人工确认，日志中可提取 patch_event_count=1、first_pass_success=False、replan_event_count=0 等指标。

当局部修补仍不足以恢复任务时，系统会转入后缀重规划或止损路径。后缀重规划尽量保留已完成前缀，只替换失败之后的步骤；terminal_stop 作为终止型候选进入 HITL 确认，并在接受后进入 FAILED 终态。

安全边界验证覆盖输入、步骤和输出阶段的风险处理。focused tests 确认 forbidden motif 在 pre-step 阶段可以触发 block 并阻止工具调用；warn 场景允许继续执行但记录风险标记。批量实验中的安全探测任务未充分触发 safety block，因此安全机制的可达性主要由确定性测试支撑，不能扩大为对所有生物安全风险的自动判定能力。

6.7 端到端与工具链验证

端到端验证由 TC-S09 和 TC-S11 覆盖，关注从任务输入到 DesignResult 的成功路径以及工具链 I/O 边界。t8 四组 smoke run 与 t9 四组 clean run 共覆盖 20 次运行，涉及 denovo、sequence evaluation、patchable 和 safety 等任务类型，均以 DONE 终态完成。

端到端验证说明，API、状态机、HITL、快照、恢复和工具适配等机制可

以在完整任务中协同工作。第七章的 84-run 策略对比实验建立在这一工程验证基础上，因此实验差异可以优先从策略配置解释，而不是归因于基本系统功能失效。

6.8 本章小结

本章围绕系统功能正确性和工程可用性，压缩整理了 13 个测试用例和六类证据材料。结果表明，系统在 API 服务、任务录入、候选生成、HITL 决策、FSM、快照恢复、前端入口、CLI 核心命令、失败恢复、安全边界和端到端执行方面均有可追溯证据。

测试边界同样需要保留：TC-S08 为部分通过，前端截图不能等同于全面浏览器兼容性测试，安全 block 路径主要由确定性 **focused tests** 支撑。在本文测试范围内，系统可以承载任务创建、运行时控制、人工决策、工具执行、恢复处理和审计追踪，为后续 CEBRA-WP 策略消融提供工程基础。

第7章 实验与结果分析

第六章已经验证系统在任务创建、状态控制、HITL、快照恢复、工具执行和审计追踪等方面具备可运行的工程基础。本章继续分析 CEBRA-WP 相关策略在批量蛋白质设计工作流中的行为差异。实验目标限定在工作流层的规划、运行时观测、恢复控制和成本控制，不涉及候选蛋白的湿实验功能验证。

本章围绕四个研究问题展开：

- （1） RQ-A1: CEBRA-WP 的运行时状态、候选重排和动作效用链路是否可执行、可追踪。
- （2） RQ-A2: 相比静态单链和固定阈值门控，动态观测是否改变系统行为。
- （3） RQ-A3 : Lite belief-state / 轻量信念状态相比 dynamic_no_belief_state 是否提供额外机制信息。
- （4） RQ-A4: 失败和恢复案例是否能由事件日志、快照和候选 metadata 还原。

7.1 实验设计

实验采用 static_top1、fixed_threshold_gate、dynamic_no_belief_state 和 lite_belief_state 四组内部消融。四组按运行时介入深度递进：静态 Top-1、固定阈值门控、无显式 belief-state 的动态观测，以及启用 RuntimeState、runtime adjustment 和 action utility 的 lite_belief_state。

图 7-1 概括了实验矩阵：任务集提供难度、预算和失败压力；策略组隔离静态选择、阈值门控、动态恢复和 Lite belief-state；指标关注成功率、首次成功、高代价调用、恢复事件和机制可观测性；证据产物包括 run config、event log、snapshot、report 和聚合 CSV。

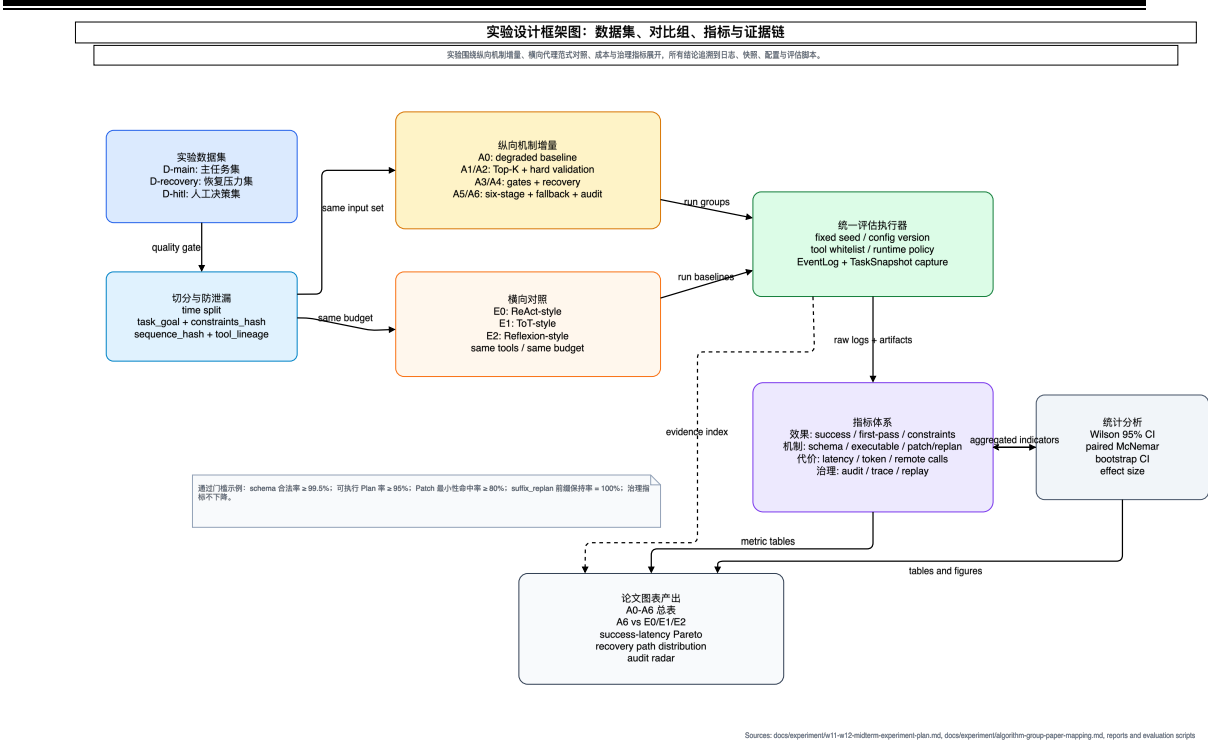


图 7-1 实验设计框架

图 7-1 的作用是限定本章统计结果的来源。后续表 7-1 至表 7-6 均来自 thesis-final-v1-001 实验矩阵及其配套日志、快照和聚合产物；对应内部证据入口为 ../thesis-project.dev/docs/experiment/thesis-final-v1-results.md 和 ../thesis-project.dev/output/experiment/thesis-final-matrix/thesis-final-v1-001/。

表 7-1 实验矩阵配置表

项目	配置
run_id	thesis-final-v1-001
freeze_id	issue209-baseline-freeze-20260326
planner_provider	deepseek-v4-pro
任务覆盖	12 个 task_keys，覆盖 T1 至 T8 共 8 类场景
策略组	static_top1 / fixed_threshold_gate / dynamic_no_belief_state / lite_belief_state
repeats	low_cost_first=2, standard=2, high_cost_sensitive=1

表 7-1（续表）

项目	配置
总 runs	84 runs, 每组 21 runs
终态结果	81 DONE, 3 FAILED
执行时间	约 6 小时
产物路径	../thesis-project.dev/output/experiment/thesis-final-matrix/thesis-final-v1-001/

任务集覆盖 8 类场景：简单 de novo 设计、序列评估、多约束稳定性优化、高代价结构预测、可修复参数失败、远程服务降级、结构性重规划和安全探测。涉及的公开结构包括 Trp-cage、Villin HP35、GB1、Ubiquitin、Top7 和 de novo oligomer。任务定义与分层依据来自 ../thesis-project.dev/docs/experiment/final-thesis-experiment-design.md 和 ../thesis-project.dev/docs/experiment/thesis-final-v1-results.md。该任务集使实验同时包含低成本成功路径、中等压力路径和高代价结构预测路径。

7.2 指标定义

本章使用五类指标。任务完成指标包括 success_rate、first_pass_success_rate、schema_valid_rate 和 executable_plan_rate。恢复行为指标包括 patch_events_mean、replan_events_mean 和 suffix_replan_events_mean。成本指标包括 high_cost_call_mean、high_cost_call_total 和 duration_ms_mean，其中高代价调用在本实验中主要对应结构预测与结构精修。机制可观测性指标包括 runtime_state_observable_rate、runtime_state_summary、budget_pressure source 和 action_utility_source。增量指标使用相邻策略组的 paired delta。

需要说明的是，本文所说的“准确度”不是候选蛋白在湿实验中的功能准确率，而是 workflow 层面的执行准确性和契约准确性，主要由 success_rate、first_pass_success_rate、schema_valid_rate 和 executable_plan_rate 表示。速度

指标主要使用 `duration_ms_mean` 描述，成本指标则用 `high_cost_call_total` 和 `high_cost_call_mean` 描述。这样处理能够在不增加新实验的前提下，从已有 84 runs 中同时观察系统是否做对、是否少返工、是否避免不必要的高代价调用以及运行耗时。

局部修补/重规划事件来自 `event log`，表示实际执行过的恢复动作；`action utility` 来自候选 `metadata`，表示算法对 `continue`、`patch_local`、`suffix_replan` 和 `stop` 等动作的评分。两者含义不同，后文表格分别列示。

7.3 四组消融主结果

表 7-2 汇总四组策略的核心指标。每组 21 runs, `static_top1` 全部 DONE，其余三组各 1 个 FAILED。

表 7-2 四组消融主实验结果

指标	<code>static_top1</code>	<code>fixed_threshold_g</code> ate	<code>dynamic_no_belief</code> _state	<code>lite_belief_st</code> ate
runs	21	21	21	21
DONE	21	20	20	20
FAILED	0	1	1	1
success_rate	1.0000	0.9524	0.9524	0.9524
first_pass_success_rate	1.0000	0.9048	0.9524	0.9524
schema_valid_rate	1.0000	1.0000	0.9524	1.0000
executable_plan_rate	1.0000	0.9524	1.0000	1.0000
waiting_chain_complete_rate	1.0000	1.0000	1.0000	1.0000
failure_traceable_rate	1.0000	1.0000	1.0000	1.0000

表 7-2（续表）

指标	static_top1	fixed_threshold_gate	dynamic_no_belief_state	lite_belief_state
runtime_state_observable_rate	0.0000	0.0000	0.0000	1.0000
patch_events_mean	0.0000	0.2857	0.0000	0.0000
replan_events_mean	0.0000	0.0000	0.0000	0.0000
suffix_replan_events_mean	0.0000	0.0000	0.0000	0.0000
high_cost_call_mean	1.0000	1.3333	0.9524	0.9524
high_cost_call_total	21	28	20	20
duration_ms_mean	241,905	300,238	226,571	272,095
action_continue_mean	1.0952	1.5238	0.9524	0.9524
action_patch_local_mean	0.0000	0.2857	0.0000	0.0000

表 7-2 的结果可以概括为三点。第一，static_top1 在该矩阵中成功率最高，说明多数初始候选已经具有较强可执行性。第二，fixed_threshold_gate 是唯一触发真实局部修补的组，平均局部修补次数为 0.2857，高代价调用总数也升至 28。第三，只有 lite_belief_state 组的 runtime_state_observable_rate 达到 1.0000，说明 Lite belief-state / 轻量信念状态链路在所有 run 中都产生了可追踪输出。

从 workflow 准确度看，四组策略的 success_rate 分别为 1.0000、0.9524、0.9524 和 0.9524；schema_valid_rate 中 static_top1、fixed_threshold_gate 和 lite_belief_state 为 1.0000，dynamic_no_belief_state 为 0.9524；executable_plan_rate 中除 fixed_threshold_gate 为 0.9524 外，其余三组为 1.0000。上述结果说明，在当前任务矩阵内，系统大多数情况下能够生成可执

行、字段有效并能结束到明确终态的工作流，但仍会暴露候选 I/O 闭包、局部修补循环等边界问题。

7.4 分层结果与机制增量

表 7-3 按任务难度和预算层级汇总成功率。3 个 FAILED run 全部位于 medium/standard 层。

表 7-3 按任务难度与预算分层的成功率

分层	runs	DONE	FAILED	success_rate
easy	32	32	0	1.0000
medium	40	37	3	0.9250
hard	12	12	0	1.0000
low_cost_first	32	32	0	1.0000
standard	40	37	3	0.9250
high_cost_sensitive	12	12	0	1.0000

分层结果说明，失败主要来自中等难度、标准预算任务。具体任务中，t2_ubiquitin_sequence_eval 贡献 2 个 FAILED，t3_gb1_stability_optimization 贡献 1 个 FAILED。hard 层样本量较小，该行仅作为本矩阵中的观察事实。

表 7-4 给出相邻策略组的 paired delta。delta 为后一个策略组减前一个策略组。

表 7-4 机制增量配对对比

对比	指标	delta	样本数	结果含义
static→fixed	success_rate	-0.0476	21	fixed 组多 1 个失败。
static→fixed	first_pass_success_rate	-0.0952	21	fixed 组首次成功率下降。
static→fixed	patch_event_count	+0.2857	21	fixed_threshold_gate 组触发真实局部修补。
static→fixed	high_cost_call_count	+0.3333	21	fixed 组高代价调用增加。
fixed→dynamic	high_cost_call_count	-0.3810	21	dynamic 组高代价调用低于 fixed。
fixed→dynamic	duration_ms	-73,667	21	dynamic 组平均耗时低于 fixed。
dynamic→lite	schema_valid_rate	+0.0476	21	lite 组 schema valid 恢复到 1.0000。
dynamic→lite	high_cost_call_count	0.0000	21	两组高代价调用均值相同。
dynamic→lite	duration_ms	+45,524	21	lite 组平均耗时高于 dynamic。

表 7-4 表明，fixed_threshold_gate 引入了可观测的局部修补行为，同时带来额外高代价调用。dynamic_no_belief_state 相比 fixed_threshold_gate 降低了高代价调用和耗时。lite_belief_state 相比 dynamic_no_belief_state 的主要差异，则体现在 schema valid、RuntimeState 和 action utility 等机制信息上。

7.5 成本与恢复行为分析

表 7-5 单独列出高代价调用和运行时间。fixed_threshold_gate 的高代价调用总数为 28，其中结构映射 27 次，结构精修 1 次；dynamic_no_belief_state

和 `lite_belief_state` 均为 20 次。

表 7-5 高代价调用与运行时间对比

组	high_cost_total	high_cost_mean	结构映射	结构精修	平均耗时
static_top1	21	1.0000	21	0	241,905 ms
fixed_threshold_gate	28	1.3333	27	1	300,238 ms
dynamic_no_belief_state	20	0.9524	20	0	226,571 ms
lite_belief_state	20	0.9524	20	0	272,095 ms

由表 7-5 可计算得出，与 `fixed_threshold_gate` 相比，`dynamic_no_belief_state` 和 `lite_belief_state` 的高代价调用总数从 28 降至 20，降幅为 28.6%。与 `static_top1` 相比，两组高代价调用总数从 21 降至 20，差异来自各自 1 个失败 run 在高代价结构预测前终止。运行时间上，`dynamic_no_belief_state` 最短，`fixed_threshold_gate` 最长，`lite_belief_state` 介于二者之间。

从速度与成本看，`dynamic_no_belief_state` 的平均耗时为 226,571 ms，是四组中最低；`static_top1` 为 241,905 ms；`lite_belief_state` 为 272,095 ms；`fixed_threshold_gate` 为 300,238 ms。`fixed_threshold_gate` 的耗时较高，与其触发 6 次 tool-level patch 和 28 次高代价调用相一致。`lite_belief_state` 虽未降低成功率层面的失败数量，但在保持 20 次高代价调用的同时产生 RuntimeState 和动作效用记录，体现出解释性与额外计算之间的取舍。

恢复事件方面，四组均未产生真实重规划；真实局部修补只出现在 `fixed_threshold_gate` 组，共 6 次 tool-level patch。说明该矩阵对局部修补形成压力，但重规划触发不足；因此批量结论主要围绕局部修补、高代价调用和机制可观测性展开。

7.6 轻量信念状态可观测性

表 7-6 对比 Lite belief-state 与其余三组的机制可观测性。该表对应 RQ-A1 和 RQ-A3。

表 7-6 轻量信念状态可观测性对比

特征	static_top1 / fixed_threshold_gate / dynamic_no_belief_state	lite_belief_state
runtime_state_summary	无有效运行时状态摘要	21/21 runs 产生 RuntimeState
runtime_state_observable_rate	0.0000	1.0000
belief-state 核心字段	未观测	p_{succ} 、 p_{sf} 、 r_{rec} 、 c_{rem} 、 e_{suf}
budget_pressure source	default	observed
action_utility_source	missing	computed
action_utilities	空对象或不可用	continue、patch_local、 suffix_replan、stop 均有 utility
high_cost_call_mean	static=1.0000, fixed=1.3333, dynamic=0.9524	0.9524

Lite belief-state / 轻量信念状态的核心证据是 21/21 runs 均产生有效 RuntimeState, 且 budget pressure 来自运行时观测而非默认回退。该结果说明, CEBRA-WP 的运行时状态更新、预算压力估计和动作效用计算均已在矩阵实验中执行。与 dynamic_no_belief_state 相比, lite_belief_state 的 success_rate 和 high_cost_call_mean 相同, 其增量主要体现为完整的 RuntimeState 和动作效用记录。

7.7 典型失败案例

如图 7-2 所示, fixed_threshold_gate 和 lite_belief_state 的恢复控制路径不同。fixed_threshold_gate 主要体现为运行时门控后的局部修补;

lite_belief_state 则在候选评估阶段产生 RuntimeState、runtime adjustment 和 action utility，并据此影响候选排序。

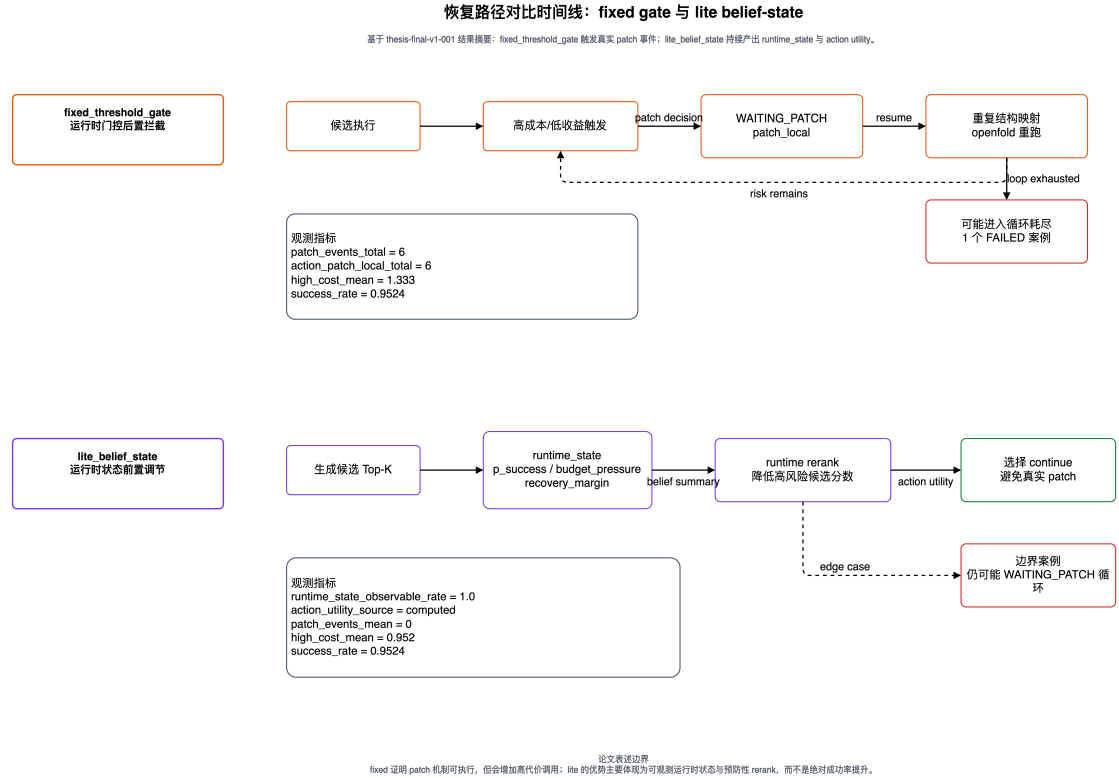


图 7-2 固定修补与 Lite belief-state 重排序的恢复路径对比

图 7-2 主要用于辅助理解三个失败案例：fixed_threshold_gate 暴露的是局部修补循环耗尽问题；lite_belief_state 展示了运行时状态可观测但仍未打破循环的情况；dynamic_no_belief_state 则对应候选 I/O 闭包校验失败。

实验中共有 3 个 FAILED runs，分别来自 fixed_threshold_gate、lite_belief_state 和 dynamic_no_belief_state 三个策略组。三个失败案例均保留了完整 event log 与 snapshot，因此可以作为可追溯的边界样本，用于分析机制行为和失败来源。

fixed_threshold_gate 组中，t2_ubiquitin_sequence_eval 的 r02 出现 auto decision loop exhausted。固定阈值门控持续触发 WAITING_PATCH，系统多次执行 patch_local 后仍满足相同门控条件，最终耗尽自动决策循环预算。该案

例说明，固定阈值门控能暴露恢复路径，也可能带来循环和额外高代价调用。

lite_belief_state 组中，t2_ubiquitin_sequence_eval 的 r01 同样出现 auto decision loop exhausted。该样本持续生成 RuntimeState，并记录 budget_pressure 升至 1.5 和 runtime adjustment 触发过程，但仍未打破 WAITING_PATCH 循环。它证明 belief-state 链路可观测，也说明恢复效果仍受候选质量和循环控制限制。

dynamic_no_belief_state 策略组中，任务 t3_gbl_stability_optimization 的 r01 出现 CANDIDATE_IO_CLOSURE_BROKEN 错误。系统在候选验证阶段发现部分 step 的输出字段不能被后续 step 正确引用，因此在执行前就把该 plan 判定为不可执行。该案例说明候选可执行性校验可以提前阻断结构无效的 plan，避免后续工具调用进入错误状态；它也对应表 7-2 中 schema_valid_rate=0.9524 的来源。

7.8 证据产物与本章结论

本章所有核心结论均可回溯到矩阵报告、CSV 聚合、run 级结果、事件日志或快照。

本章实验得到四点结论：CEBRA-WP 机制链路可执行、可追踪；lite_belief_state 组 21/21 runs 产生 RuntimeState；fixed_threshold_gate 触发 6 次真实局部修补，高代价调用总数达到 28；3 个 FAILED run 均有可追溯事件链，主要暴露候选验证、局部修补循环和运行时控制边界。

在当前 thesis-final-v1-001 设置下，实验支持“CEBRA-WP 机制已实现且可观测”、“固定阈值门控恢复存在额外成本”、“Lite belief-state / 轻量信念状态提供运行时决策解释信息”等结论。成功率方面，static_top1 为 1.0000，其余三组为 0.9524，最终成功率提升并非本章的主要实验结论。

结论将结合上述实验结论与边界，总结本文贡献并讨论后续改进方向。

结 论

本文面向蛋白质设计 workflow 中的工具异构、运行时失败和人工审查需求，设计并实现了一个大模型驱动的多 Agent 原型系统。系统在已有工具和远程模型服务之上，完成任务接入、候选计划生成、工具执行、HITL 决策、快照恢复、事件审计和结果报告，把分散工具组织为可规划、可执行、可恢复、可追溯的工作流。本文工作量主要体现在多入口、多模块和多证据链的打通，而不是单一模型调用；结论限定在工作流层和工程原型层，不等同于湿实验验证。

在系统设计与实现方面，本文将蛋白质设计任务抽象为受目标、约束、工具能力、I/O 契约、预算和运行时观测共同影响的工作流规划问题。系统划分 PlannerAgent、ExecutorAgent、SafetyAgent 和 SummarizerAgent 等职责；通过 ProteinToolKG 描述工具能力和 I/O 兼容关系，通过 FSM 约束任务状态，通过 PendingAction、Decision、EventLog 和 TaskSnapshot 支撑人在环确认与审计。工程实现形成后端 API、前端工作台、运行时执行器和工具适配层。

在算法与验证方面，本文提出并实现 CEBRA-WP，将约束感知、证据感知、Lite belief-state / 轻量信念状态和恢复自适应动作统一到工作流层。算法不替代底层蛋白质设计模型，而是在候选筛选、运行时重排序和恢复动作选择中综合 schema、成本、风险、证据充分性和预算压力。系统测试覆盖 13 个用例，其中 12 个通过、1 个 CLI 相关用例部分通过；84 runs 批量消融实验得到 81 个 DONE 和 3 个 FAILED。工作流准确度方面，各组 success_rate 为 0.9524 至 1.0000，schema_valid_rate 和 executable_plan_rate 大多达到 1.0000；速度方面，平均耗时约 226,571 ms 至 300,238 ms。lite_belief_state 组 21/21 runs 产生 RuntimeState, fixed_threshold_gate 触发 6 次真实局部修补，说明 CEBRA-WP 机制链路可执行、可观测。

本文的主要贡献可以概括为四点：第一，构建面向蛋白质设计任务的可恢复、可审计多 Agent 工作流原型，打通任务接入、候选计划、工具执行、人工

决策和报告输出；第二，提出 CEBRA-WP，把工具知识、硬可行性过滤、运行时状态和恢复动作效用纳入同一决策过程；第三，将 FSM、HITL、快照和事件日志用于约束 Agent 行为，缓解任务状态、人工决策和工具失败容易脱节的问题；第四，建立系统测试与内部消融证据链，使关键结论能够回溯到测试用例、事件日志和实验聚合结果。

本文仍存在局限。首先，实验矩阵有限，每组 21 runs 只能支撑机制分析。其次，dynamic_no_belief_state 与 lite_belief_state 在 success_rate 和 high_cost_call_mean 上相同，任务集对二者差异的放大能力不足。再次，84-run 矩阵中真实局部修补只出现在 fixed_threshold_gate 组，四组均未触发真实后缀重规划；suffix_replan、terminal_stop 和 safety block 主要由 focused tests 支撑。最后，系统仍处于原型阶段，持久化、ProteinToolKG 动态更新和远程服务探活等方面需完善。

后续工作可以从三个方向推进：一是扩大任务规模，引入工具不可用、预算冲突、schema 错误、I/O 闭包错误和安全约束冲突等压力任务，使 patch_local、suffix_replan、terminal_stop 和 safety block 在批量矩阵中均可观察；二是完善数据库持久化、ProteinToolKG 动态更新、事件日志追踪、前端结构可视化和远程服务配额管理；三是加入外部 Agent 方法和蛋白质设计前沿方法作为比较对象，检验 CEBRA-WP 在结构化约束、HITL、恢复审计和高代价控制方面的相对价值。

参考文献

- [1] KSHIRSAGAR M, NIE A, CHENG C A, 等. RosettaSearch: Multi-Objective Inference-Time Search for Protein Sequence Design[A/OL]. arXiv, 2026[2026-05-11]. <http://arxiv.org/abs/2604.17175>. DOI:10.48550/arXiv.2604.17175.
- [2] JUMPER J, EVANS R, PRITZEL A, 等. Highly accurate protein structure prediction with AlphaFold[J/OL]. Nature, 2021, 596(7873): 583-589. DOI:10.1038/s41586-021-03819-2.
- [3] ABRAMSON J, ADLER J, DUNGER J, 等. Accurate structure prediction of biomolecular interactions with AlphaFold 3[J/OL]. Nature, 2024, 630(8016): 493-500. DOI:10.1038/s41586-024-07487-w.
- [4] FERRUZ N, SCHMIDT S, HÖCKER B. ProtGPT2 is a deep unsupervised language model for protein design[J/OL]. Nature Communications, 2022, 13(1): 4348. DOI:10.1038/s41467-022-32007-7.
- [5] DAUPARAS J, ANISHCHENKO I, BENNETT N, 等. Robust deep learning-based protein sequence design using ProteinMPNN[J/OL]. Science, 2022, 378(6615): 49-56. DOI:10.1126/science.add2187.
- [6] MANFREDI M, VAZZANA G, SAVOJARDO C, 等. AlphaFold2 and ESMFold: A large-scale pairwise model comparison of human enzymes upon Pfam functional annotation[J/OL]. Computational and Structural Biotechnology Journal, 2025, 27: 461-466. DOI:10.1016/j.csbj.2025.01.008.
- [7] WATSON J L, JUERGENS D, BENNETT N R, 等. De novo design of protein structure and function with RFdiffusion[J/OL]. Nature, 2023, 620(7976): 1089-1100. DOI:10.1038/s41586-023-06415-8.
- [8] YAO S, ZHAO J, YU D, 等. ReAct: Synergizing Reasoning and Acting in Language Models[A/OL]. arXiv, 2023[2026-05-11]. <http://arxiv.org/abs/2210.03629>. DOI:10.48550/arXiv.2210.03629.

- [9] SCHICK T, DWIVEDI-YU J, DESSÌ R, 等. Toolformer: Language Models Can Teach Themselves to Use Tools[A/OL]. arXiv, 2023[2026-05-11]. <http://arxiv.org/abs/2302.04761>. DOI:10.48550/arXiv.2302.04761.
- [10] AHDRITZ G, BOUATTA N, FLORISTEAN C, 等. OpenFold: retraining AlphaFold2 yields new insights into its learning mechanisms and capacity for generalization[J/OL]. Nature Methods, 2024, 21(8): 1514-1524. DOI:10.1038/s41592-024-02272-z.
- [11] DI TOMMASO P, CHATZOU M, FLODEN E W, 等. Nextflow enables reproducible computational workflows[J/OL]. Nature Biotechnology, 2017, 35(4): 316-319. DOI:10.1038/nbt.3820.
- [12] BELLMAN R. A Markovian Decision Process[J/OL]. Indiana University Mathematics Journal, 1957, 6(4): 679-684. DOI:10.1512/iumj.1957.6.56038.
- [13] PUTERMAN M L. Markov Decision Processes: Discrete Stochastic Dynamic Programming[M/OL]. 1 版. Wiley, 1994[2026-05-11]. <https://onlinelibrary.wiley.com/doi/book/10.1002/9780470316887>. DOI:10.1002/9780470316887.
- [14] SONDIK E J. The Optimal Control of Partially Observable Markov Processes over the Infinite Horizon: Discounted Costs[J/OL]. Operations Research, 1978, 26(2): 282-304. DOI:10.1287/opre.26.2.282.
- [15] MONAHAN G E. State of the Art—A Survey of Partially Observable Markov Decision Processes: Theory, Models, and Algorithms[J/OL]. Management Science, 1982, 28(1): 1-16. DOI:10.1287/mnsc.28.1.1.
- [16] KAEHLBLING L P, LITTMAN M L, CASSANDRA A R. Planning and acting in partially observable stochastic domains[J/OL]. Artificial Intelligence, 1998, 101(1-2): 99-134. DOI:10.1016/S0004-3702(98)00023-X.
- [17] LEAVER-FAY A, TYKA M, LEWIS S M, 等. Rosetta3[M/OL]//Methods in Enzymology: 卷 487. Elsevier, 2011: 545-574[2026-05-11].

- <https://linkinghub.elsevier.com/retrieve/pii/B9780123812704000196>. DOI:10.1016/B978-0-12-381270-4.00019-6.
- [18] COCK P J A, ANTAO T, CHANG J T, 等. Biopython: freely available Python tools for computational molecular biology and bioinformatics[J/OL]. *Bioinformatics*, 2009, 25(11): 1422-1423. DOI:10.1093/bioinformatics/btp163.
- [19] DEELMAN E, SINGH G, SU M H, 等. Pegasus: A Framework for Mapping Complex Scientific Workflows onto Distributed Systems[J/OL]. *Scientific Programming*, 2005, 13(3): 219-237. DOI:10.1155/2005/128026.
- [20] YAO S, YU D, ZHAO J, 等. Tree of Thoughts: Deliberate Problem Solving with Large Language Models[A/OL]. *arXiv*, 2023[2026-05-11]. <http://arxiv.org/abs/2305.10601>. DOI:10.48550/arXiv.2305.10601.
- [21] SHINN N, CASSANO F, BERMAN E, 等. Reflexion: Language Agents with Verbal Reinforcement Learning[A/OL]. *arXiv*, 2023[2026-05-11]. <http://arxiv.org/abs/2303.11366>. DOI:10.48550/arXiv.2303.11366.
- [22] XIE T, ZHANG D, CHEN J, 等. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments[A/OL]. *arXiv*, 2024[2026-05-11]. <http://arxiv.org/abs/2404.07972>. DOI:10.48550/arXiv.2404.07972.
- [23] GE F, ZHU J, ZHANG L, 等. AutoBinder Agent: An MCP-Based Agent for End-to-End Protein Binder Design[A/OL]. *arXiv*, 2026[2026-05-11]. <http://arxiv.org/abs/2602.00019>. DOI:10.48550/arXiv.2602.00019.
- [24] HOU X, LIU J, SHI C, 等. Property-driven Protein Inverse Folding With Multi-Objective Preference Alignment[A/OL]. *arXiv*, 2026[2026-05-11]. <http://arxiv.org/abs/2603.06748>. DOI:10.48550/arXiv.2603.06748.
- [25] CARRARA N, LEURENT E, LAROCHE R, 等. Budgeted Reinforcement Learning in Continuous State Space[A/OL]. *arXiv*, 2019[2026-05-11]. <http://arxiv.org/abs/1903.01004>. DOI:10.48550/arXiv.1903.01004.

- [26]SHANI G. Heuristics for Partially Observable Stochastic Contingent Planning[A/OL].
arXiv, 2024[2026-05-11]. <http://arxiv.org/abs/2410.05870>.
DOI:10.48550/arXiv.2410.05870.
- [27]XIONG J, GAUR I, LUKARSKA M, 等. ProteinGuide: On-the-fly property guidance for
protein sequence generative models[A/OL]. arXiv, 2026[2026-05-11].
<http://arxiv.org/abs/2505.04823>. DOI:10.48550/arXiv.2505.04823.
- [28]WANG Z, FAN J, GUO R, 等. ProteinZero: Self-Improving Protein Generation via Online
Reinforcement Learning[A/OL]. arXiv, 2026[2026-05-11]. <http://arxiv.org/abs/2506.07459>.
DOI:10.48550/arXiv.2506.07459.

附录 A 系统验证与测试明细

本附录补充第六章“系统测试与验证”中的必要测试用例明细。正文第六章只保留测试策略、覆盖矩阵和关键验证结论；本附录列出证据链路、材料范围以及 TC-S01 至 TC-S13 的设计目的、触发方式、预期结果、证据编号和结论边界，便于答辩复核。

本附录中的证据编号与开发目录 `../thesis-project.dev/docs/system-validation/evidence-index.md`、`test-case-table.md` 保持一致。证据编号只说明材料类型和可追溯位置，不作为论文正文图号，也不重复正文实验结论。

A.1 证据链路与材料范围

表 A-1、表 A-2 和表 A-3 分别列出系统验证证据链路、证据编号规则和材料目录，用于说明附录材料与正文验证结论之间的对应关系。

表 A-1 系统验证证据链路

设计	执行	归档	追踪	论文使用
测试用例设计	执行入口	证据归档	状态追踪	论文结论
TC-S01 至 TC-S13	API / Web / CLI / pytest / 实验运行	EVD-API、EVD-TEST、EVD-CLI、FIG-SV、EVD-LOG、EVD-EXP	EventLog / Snapshot / Report	正文只使用可被证据支撑的边界化表述

表 A-2 系统验证证据编号规则

证据前缀	证据类型	数量/范围	主要内容	论文使用方式
EVD-API	API 响应 JSON	8 项	/health、/capabilities/readiness、任务详情、事件、报告和 pending action 接口响应	支撑 API 合约、任务状态和事件可追溯性。
EVD-TEST	pytest 执行日志	4 组	API/Web smoke、FSM/HITL/快照、安全恢复、CLI 测试结果	支撑自动化测试通过情况和异常边界。
EVD-CLI	CLI 输出日志	4 项	intake schema、task show、timeline show、report show 输出	支撑 CLI 主路径可用和部分命令限制。
FIG-SV	前端验证截图	18 张	Dashboard、Task Builder、Task Detail、Event Timeline 页面截图	作为系统验证截图证据，不作为正文正式图号。

表 A-2（续表）

证据前缀	证据类型	数量/范围	主要内容	论文使用方式
EVD-LOG	EventLog Snapshot / Report 样本	/ 8 组	等待态、决策、恢复、terminal_stop、报告产物和快照样本	支撑 FSM、HITL、恢复和审计链路。
EVD-EXP	实验矩阵与端到端聚合	4 项	smoke run 、 clean run 、 action distribution、实验聚合结果	支撑端到端流程、工具链执行和第七章实验基础。

表 A-3 系统验证材料目录

材料类别	主要位置	用途
系统验证索引	../thesis-project.dev/docs/system-validation/evidence-index.md	统一登记 EVD-API、EVD-TEST、EVD-CLI、FIG-SV、EVD-LOG、EVD-EXP 的文件路径和说明。
测试用例表	../thesis-project.dev/docs/system-validation/test-case-table.md	记录 TC-S01 至 TC-S13 的用例编号、覆盖验证点和执行结果。
验证检查清单	../thesis-project.dev/docs/system-validation/system-validation-checklist.md	记录系统验证项、检查结果和剩余限制。
前端截图	../thesis-project.dev/docs/system-validation/06-ui-screenshots/	保存 Dashboard、Task Builder、Task Detail 和 Timeline 页面截图。
日志与快照样本	../thesis-project.dev/data/logs/、../thesis-project.dev/data/snapshots/	保存任务事件、运行时快照和恢复链路证据；固定样本复制于 docs/system-validation/04-data-consistency/。
实验聚合结果	../thesis-project.dev/docs/experiment/ 及实验输出目录	保存 smoke/clean run 和最终实验矩阵的聚合结果。

A.2 测试用例设计明细

表 A-4 和表 A-5 列出 TC-S01 至 TC-S13 的测试用例明细，正文仅引用其覆盖范围和结论边界。

表 A-4 系统测试用例设计明细（TC-S01 至 TC-S07）

用例	设计目的与触发	预期结果	实际结果与证据	结论边界
TC-S01 环境与能力就绪	检查 API 服务启动状态，并观察系统是否能实时暴露工具能力。触发方式为访问 /health 与 /capabilities/readiness。	/health 应返回 status=ok; readiness 应列出各工具能力的状态、错误类别和恢复建议。	结果通过：/health 给出任务数、工具数量和路径信息；readiness 返回 15 条能力记录，并分别说明 ready、degraded、unavailable 状态。证据：EVD-API-01、EVD-API-02。	该项说明当前环境具备能力可观测性，但不表示所有外部工具都已安装或可用。
TC-S02 API 合约与任务录入	验证 Task Intake、任务创建、任务详情、事件和报告接口的契约边界。触发 schema、草稿、confirm、互斥创建模式和典型查询接口。	schema 可返回；缺字段或互斥模式被拒绝；DONE 任务可读取报告；DONE 任务没有 pending actions；事件 API 可读取磁盘日志。	结果通过：API 自动化测试为 71 passed；schema 能返回完整字段注册表；错误请求会给出明确异常；真实实验任务的 events 可从日志读取。证据：EVD-TEST-01、EVD-API-03 至 EVD-API-08。	该项支持 API 合约和主要异常边界可用，但不等同于已经穷尽未来所有字段组合。
TC-S03 计划候选生成	检查 PlannerAgent 生成的候选计划是否带有必要契约字段，同时确认其职责边界。	候选需要包含 candidate_id、评分分解、风险、成本、解释和来源引用；Planner 不执行工具，也不直接改变任务状态。	通过。focused suite 107 passed；候选字段完整，低置信度场景可生成 PendingAction(plan_confirm)。证据：EVD-TEST-02、EVD-LOG-04。	证明候选契约完整，不证明候选在所有科学任务上均为最优。
TC-S04 HITL 决策一致性	验证 PendingAction 与 Decision 的绑定、一次性决策和等待态执行停止。	合法决策被应用；缺字段返回 400；重复决策返回冲突；错误绑定被拒绝；等待态不继续调用工具。	通过。PendingAction 创建、决策应用和等待退出均有事件记录；前端候选/决策区域可展示。证据：EVD-TEST-02、FIG-SV-15、EVD-LOG-08。	证明 HITL 运行时契约正确，不评价人工决策本身是否科学最优。

表 A-4（续表）

用例	设计目的与触发	预期结果	实际结果与证据	结论边界
TC-S05 FSM 状态迁移	验证有限状态机的合法迁移、非法迁移拒绝和终态不可变性。	合法迁移成功；非法迁移被拒绝；终态不可变； terminal_stop 接受后进入 FAILED 并保留审计链。	通过。状态迁移测试覆盖关键路径； terminal_stop 事件链可追溯。证据：EVD-TEST-02、EVD-LOG-03。	证明状态控制规则有效，不代表运行中永远不会出现外部进程中断。
TC-S06 快照持久化与恢复	验证进入等待态前是否保存 PendingAction、事件和 TaskSnapshot，并确认恢复后不自动推进。	进入等待态前写入 pending action、事件和快照；恢复后仍处于等待态； runtime state 保存在 artifacts 中。	通过。快照包含 pending_action_id、completed_step_ids、plan_version 和 artifacts.runtime_state。证据：EVD-TEST-02、EVD-LOG-01 至 EVD-LOG-03。	证明本地快照恢复语义正确，不等同于分布式容灾或多节点一致性。
TC-S07 Web 前端可用性	验证 Dashboard、Task Builder、Task Detail 和 Event Timeline 是否能展示核心状态和交互入口。	页面正常加载；任务状态、pending action、事件和报告信息与 API 保持一致。	通过。Dashboard、Task Builder、Task Detail、Timeline 共 18 张截图； Web smoke test 通过。证据：FIG-SV-01 至 FIG-SV-18、EVD-TEST-01。	截图和 smoke test 证明关键页面可用，不代表全面浏览器兼容性和无障碍测试。

表 A-5 系统测试用例设计明细（TC-S08 至 TC-S13）

用例	设计目的与触发	预期结果	实际结果与证据	结论边界
TC-S08 CLI 可用性	验证命令行入口是否支持 schema 查看、任务查看和报告/时间线访问。	schema 和 task show 输出有效内容；timeline/report 应能展示对应信息或暴露当前限制。	部分通过。intake schema -- json 输出完整字段注册表；task show 输出任务状态和能力 readiness；CLI 自动化测试 16 passed；timeline show 和 report show 当前仅输出 usage。证据：EVD-CLI-01 至 EVD-CLI-04、EVD-TEST-04。	CLI 主路径可用，但 timeline/report 子命令尚未完整实现；正文中必须保留“部分通过”。
TC-S09 端到端任务流程	验证从任务输入到 DONE 终态和 DesignResult 的完整路径。运行 t8 smoke run 和 t9 clean run 并检查报告接口和聚合指标。	任务完成并进入 DONE；每个 DONE 任务有 StepResult；report 返回有效结果；未完成任务请求 report 返回 404。	通过。t8 四组 smoke 和 t9 四组 clean run 共 20 次运行全部 DONE，success rate、schema valid rate 和 executable plan rate 均为 1.0。证据：EVD-EXP-01、EVD-LOG-05、EVD-API-06。	证明系统可承载端到端任务，不代表最终蛋白质具有湿实验验证的生物学功能。
TC-S10 异常输入与安全边界	验证异常输入、forbidden motif、安全 warn/block 和工具调用阻断。	block 阻止工具调用；allow 不误阻断；warn 放行但记录风险标记和安全事件。	通过。4 个确定性 focused tests 覆盖安全判定到执行阻断链路。证据：EVD-TEST-03。	证明安全机制路径可达，不宣称自动覆盖所有生物安全风险；矩阵实验中安全任务未充分触发 block。
TC-S11 工具链执行与 I/O	验证工具适配器调用、输入输出边界和候选可执行性检查。	工具调用成功；无 dummy 输入；无 provider payload validation error；无效候选在执行前被结构化拒绝。	结果通过：t8/t9 共 20 次运行中，相关工具调用均为 success；I/O 边界测试能够拒绝错误候选。证据：EVD-TEST-03、EVD-EXP-01。	该项说明当前工具适配路径有效，但不能外推到所有第三方工具版本和部署环境。

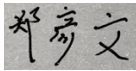
用例	设计目的与触发	预期结果	实际结果与证据	结论边界
TC-S12 失败 恢复流程	验证 retry、局 部修补、后缀重 规划的分层恢复 路径。	可重试失败先进入有界 重试；耗尽后转入恢复 逻辑；patch_local / suffix_replan 候选进入 HITL；恢复事件可统 计。	通过。确定性 retry patch 样本产生 patch_event_count=1、 first_pass_success=False、 replan_event_count=0；分 层局部修补和后缀重规划 样本可追溯。证据： EVD-TEST-03、EVD- LOG-01 至 EVD-LOG- 03。	证明恢复路径 存在并可审 计，不代表所 有失败类型都 能恢复为 DONE。
TC-S13 恢复 止损与审计 链路	验证 terminal_stop 作 为恢复动作时是 否通过 FSM/HITL 进入 FAILED，并保 留审计链。	决策应用后任务进入 FAILED 终态；事件日 志记录 WAITING_ENTER、 DECISION_APPLIED、 WAITING_EXIT 等关 键事件。	结果通过：terminal_stop 的审计链记录完整，可通 过事件接口还原。证据： EVD-TEST-02/03、EVD- LOG-03/04。	该项说明止损 路径处于受控 状态；FAILED 不一定意味着 系统错误，也 可能来自策略 性终止。

哈尔滨工业大学本科毕业论文（设计）

原创性声明和使用权限

本科毕业论文（设计）原创性声明

本人郑重声明：此处所提交的本科毕业论文（设计）《基于大模型驱动的 Agent 协作新一代蛋白质设计系统开发》，是本人在导师指导下，在哈尔滨工业大学攻读学士学位期间独立进行研究工作所取得的成果，且毕业论文（设计）中除已标注引用文献的部分外不包含他人完成或已发表的研究成果。对本毕业论文（设计）的研究工作做出重要贡献的个人和集体，均已在文中以明确方式注明。本毕业论文（设计）对使用 AI 工具的情况进行了明确标注。

作者签名：  日期：2026 年 5 月 22 日

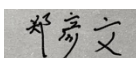
本科毕业论文（设计）使用权限

本科毕业论文（设计）是本科生在哈尔滨工业大学攻读学士学位期间完成的成果，知识产权归属哈尔滨工业大学。本科毕业论文（设计）的使用权限如下：

（1）学校可以采用影印、缩印或其他复制手段保存本科生上交的毕业论文（设计），并向有关部门报送本科毕业论文（设计）；（2）根据需要，学校可以将本科毕业论文（设计）部分或全部内容编入有关数据库进行检索和提供相应阅览服务；（3）本科生毕业后发表与此毕业论文（设计）研究成果相关的学术论文和其他成果时，应征得导师同意，且第一署名单位为哈尔滨工业大学。

保密论文在保密期内遵守有关保密规定，解密后适用于此使用权限规定。

本人知悉本科毕业论文（设计）的使用权限，并将遵守有关规定。

作者签名：  日期：2026 年 5 月 22 日

导师签名： 日期： 年 月 日

致 谢

四年的本科学习即将结束，回顾本次毕业设计与论文写作的过程，我在系统开发、问题分析、论文组织等方面都得到了许多帮助。在此，向所有给予我指导、支持与陪伴的人表示诚挚的感谢。

首先，感谢我的指导教师陈源龙老师在论文选题、研究思路、系统实现和论文写作等方面给予的耐心指导。从论文开题、中期检查到最终定稿，陈老师提出了许多宝贵的意见，帮助我不断发现问题、完善系统设计，并逐步理清论文的整体结构与表达方式，使我能够较为顺利地完成本次毕业设计工作。

其次，感谢学院各位老师在本科阶段的教学与培养。通过四年的课程学习、实验训练、课程设计与项目实践，我逐渐建立了对计算机行业和软件工程专业的基本认识，也在不断完成各类实践任务的过程中提升了分析问题、解决问题和独立学习的能力。这些知识与训练为本次毕业设计的完成奠定了重要基础。

同时，感谢同学和朋友们在学习与生活中给予我的陪伴、帮助与鼓励。在论文撰写和系统开发过程中，与你们的交流和讨论为我提供了许多启发，也让我在遇到困难和压力时能够继续调整状态、推进工作。大学生活中的许多共同经历，也将成为我非常珍惜的回忆。

最后，感谢我的家人一直以来的理解、支持与包容。正是你们在背后的关心、鼓励与支持，使我能够安心完成本科学业，并以更加坚定的心态面对未来的学习、工作与生活。

谨以此文，向所有关心和帮助过我的老师、同学、朋友和家人表示衷心的感谢。